

The Omni-Compass: source code

Owner and author: The Omni-Compass LLC

Version (git commit): 33f06327eb45

Printed: 2026-10-04

Identifying portion for deposit: the first 25 and the last 25 pages of the source listing.

366 source files, 102,621 lines, 1,835 pages of source in the full listing.

Languages: Python, C++, shell, YAML (GitHub Actions workflows).

Order: the engine (omnicompass/), the controllers (omni_controller/), the C++ twins (cpp/), the fleet and realms models, tools, scripts, deployment, tests, workflows.

Copyright (c) 2026 The Omni-Compass LLC. All rights reserved. Not open source. Any commercial use, commercialization, monetization, production use, redistribution or hosted service requires a signed, paid Omni-Compass Enterprise License. Patent applications, copyright registrations and trademark applications have been filed in the United States by The Omni-Compass LLC.

```
==== omnicompass/__init__.py (3 lines) ====
```

```
1 # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2 # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3 # Omni-Compass Enterprise License. See LICENSE.
```

```

==== omnicompass/adaptor.py (309 lines) ====
1  """Omni-Compass stack governor.
2
3  sense      : stack telemetry -> normalized observation vector
4  assimilate : observation blended into the six-state engine state
5  evolve     : one macro step of equations (1)-(7) with u = 0
6  allocate   : engine state -> directive (capacity, change gating, power, rollback, routing, alerting, paging)
7  authority  : OBSERVE | AUTOPILOT; kill() revokes authority and returns control to the native managers
8  """
9  from __future__ import annotations
10
11  import math
12  from collections import deque
13  from dataclasses import dataclass, asdict, field
14  from typing import Any, Dict, List, Optional
15
16  from .core import State, Params, macro_step, v_eff, control_command, U_AUTHORITY
17
18  OBSERVE = "observe"
19  AUTOPILOT = "autopilot"
20
21  # Engine parameters used for stack governance (fixed, not sampled).
22  STACK_PARAMS = Params(alpha=4.2, beta_int=0.0, beta_ext=0.0, k=1.7, sigma_1=0.38, delta=0.705, gamma_1=1.35,
23                        gamma_c=1.0, lambda_0=-0.40, lambda_1=1.55, lambda_2=1.15, c=1.0, E_max=2.0,
24                        omega_B=1.0, Q_B=2.5, alpha_s=0.12, beta_s=0.10, mu=3.782, alpha_E=3.487, alpha_U=4.2,
25                        lambda_I=0.719, lambda_U=0.0)
26  INITIAL_STATE = State(E=0.12, U=0.82, I_U=0.0, S=0.0, B=0.0, B_dot=0.0)
27  ASSIMILATION = 0.339
28
29
30  def clamp(v: float, lo: float, hi: float) -> float:
31      return lo if v < lo else hi if v > hi else v
32
33
34  def observe_vector(obs: Dict[str, float], conflicts: int) -> Dict[str, float]:
35      """Normalize telemetry. Inputs are the stack's delayed observations."""
36      return {
37          "q": clamp(float(obs.get("queue_ratio", 0.0)), 0.0, 2.0),
38          "load": clamp(float(obs.get("load_ratio", 0.0)), 0.0, 2.0),
39          "power": clamp(float(obs.get("power_stress", 0.0)), 0.0, 1.5),
40          "thermal": clamp(float(obs.get("thermal", 0.0)), 0.0, 1.5),
41          "network": clamp(float(obs.get("network_stress", 0.0)), 0.0, 1.5),
42          "drift": clamp(float(obs.get("drift_ratio", 0.0)), 0.0, 1.5),
43          "stale": clamp(float(obs.get("stale", 0.0)), 0.0, 1.0),
44          "security": clamp(float(obs.get("security_block", 0.0)), 0.0, 1.0),
45          "conflict": clamp(conflicts / 4.0, 0.0, 1.0),
46      }
47
48
49  def assimilate(x: State, o: Dict[str, float], p: Params) -> State:
50      """Blend the observation into the engine state and set the forcing terms."""
51      q, load, pw, th, nw, dr, st, sec, cf = (o[k] for k in
52          ("q", "load", "power", "thermal", "network", "drift", "stale", "security", "conflict"))
53      e_obs = clamp(0.25 * q + 0.18 * max(0.0, load - 0.85) + 0.16 * pw + 0.13 * th + 0.10 * nw + 0.08 * dr
54                  + 0.06 * cf + 0.04 * st, 0.0, 1.5)
55      u_obs = clamp(1.0 - (0.27 * q + 0.18 * pw + 0.16 * th + 0.12 * nw + 0.12 * dr + 0.10 * cf + 0.05 * st), 0.0, 1.0)
56      s_obs = clamp(0.36 * th + 0.28 * pw + 0.18 * nw + 0.10 * sec + 0.08 * st - 0.20 * q, -1.0, 1.0)
57      b_obs = clamp(0.52 * pw + 0.26 * th + 0.22 * dr, -1.0, 1.5)
58      i_obs = clamp(q + dr + cf, 0.0, 2.0)
59      a = ASSIMILATION
60      p.beta_int = clamp(0.58 * q + 0.42 * max(0.0, load - 0.75), 0.0, 1.0)
61      p.beta_ext = clamp(0.33 * pw + 0.27 * th + 0.20 * nw + 0.12 * st + 0.08 * sec, 0.0, 1.0)
62      return State(E=(1 - a) * x.E + a * e_obs, U=(1 - a) * x.U + a * u_obs, I_U=(1 - a) * x.I_U + a * i_obs,
63                  S=(1 - a) * x.S + a * s_obs, B=(1 - a) * x.B + a * b_obs, B_dot=(1 - a) * x.B_dot + a * (b_obs - x.B))
64
65
66  @dataclass(frozen=True)
67  class AllocationLaw:
68      """Engine state -> governing directive.
69
70      release    capacity removed only while push <= push_release (the engine's controller reports convergence)
71      add        without backlog, capacity added only while push >= push_add (the engine's controller reports need)
72      push       equation (2) control law evaluated on the evolved state with target sigma = +1:
73                u = clip(-f_U(x) + KP (1 - U), -U_AUTHORITY, U_AUTHORITY); push = u / U_AUTHORITY
74                (f_U contains the double-well drift and -(dE/dt)/E_max from equation (1))
75      headroom   rho* = clamp(rho0 - kI*I_U - kE*E, rho_min, rho0)
76      sizing     load_s = load * cap_now if size_at_full_cap else load
77      capacity   n_req = ceil(n * load / rho*) + ceil(kq * q * n); delta = clamp(n_req - n, -down_max, +up_max)
78      guard      no capacity removal while q >= guard_queue
79      band       capacity removed only when n_req <= n - down_band
80      dwell      capacity removed only after surplus persists for down_dwell intervals and no earlier than

```

```

81         down_after_add intervals after the last addition
82     envelope no capacity addition while observed power >= power_ceiling (power_ceiling_backlog while q >= guard_queue
83 )
84     security no capacity addition while a security block is observed
85     change Terraform plans only while U >= U_gate, only in the direction of the live capacity decision
86     power q < guard_queue and U >= U_gate: cap = clamp(cap_now * load * (1 + margin), cap_min, 1) (trim to deman
87 d)
88         q >= guard_queue at the power envelope: cap = max(cap_min, cap_now - spread_step) (spread)
89         q >= guard_queue below the envelope: cap held (capacity added as nodes; 1 at max_nodes)
90         otherwise cap = 1; always cap <= 1 - cap_gain * max(0, B - B_cap)
91     rollback a proposed rollback is authorised if S >= S_rollback, U is falling, or security blocks change
92     replicas direct actuation: desired = ceil(replicas * load / rho*) (the HPA formula with the engine-set target),
93         increases unless capacity is being reduced, decreases only while reducing;
94         otherwise autoscaler increases approved unless reducing, decreases only while reducing
95     consistency a step that expands capacity never tightens the power cap
96     paging none: the governor holds every action a paged operator would take
97     ""
98     rho0: float = 0.914
99     rho_min: float = 0.627
100     kI: float = 0.252
101     kE: float = 0.297
102     kq: float = 0.795
103     up_max: int = 2
104     down_max: int = 1
105     guard_queue: float = 0.05
106     power_ceiling: float = 0.95
107     power_ceiling_backlog: float = 1.00
108     spread_step: float = 0.05
109     backlog_release: bool = False
110     size_at_full_cap: bool = False
111     push_release: float = 0.005
112     push_add: float = -9.0
113     U_gate: float = 0.481
114     margin: float = 0.006
115     cap_min: float = 0.706
116     B_cap: float = 0.40
117     cap_gain: float = 0.60
118     S_rollback: float = 0.55
119     route_network: float = 0.20
120     down_band: int = 4
121     down_dwell: int = 4
122     down_after_add: int = 4
123     min_nodes: int = 8
124     max_nodes: int = 40
125
126 @dataclass
127 class Governor:
128     law: AllocationLaw = field(default_factory=AllocationLaw)
129     mode: str = AUTOPILOT
130     killed: bool = False
131     x: State = field(default_factory=lambda: State(**asdict(INITIAL_STATE)))
132     p: Params = field(default_factory=lambda: Params(**asdict(STACK_PARAMS)))
133     t: float = 0.0
134     current_cap: float = 1.0
135     nodes: int = 20
136     surplus_streak: int = 0
137     since_add: int = 99
138     last_push: float = 0.0
139     log: deque = field(default_factory=lambda: deque(maxlen=4096))
140     evolve: bool = True
141
142     def kill(self) -> None:
143         """Revoke authority; the engine continues to compute and log directives."""
144         self.killed = True
145
146     def set_mode(self, mode: str) -> None:
147         if mode not in (OBSERVE, AUTOPILOT):
148             raise ValueError(mode)
149         self.mode = mode
150
151     @property
152     def has_authority(self) -> bool:
153         return self.mode == AUTOPILOT and not self.killed
154
155     def step(self, obs: Dict[str, float], conflicts: int) -> Dict[str, Any]:
156         """Sense, assimilate, evolve, allocate."""
157         o = observe_vector(obs, conflicts)
158         U_before = self.x.U
159         xa = assimilate(self.x, o, self.p)
160         if self.evolve:

```

```

160         xn, _ = macro_step(xa, self.p, self.t, target=None)
161     else:
162         xn = xa
163         self.x = xn
164         self.t += 0.1
165         L = self.law
166         n = int(self.nodes)
167         u_push, _ = control_command(xn, self.p, self.t, +1)
168         push = u_push / U_AUTHORITY
169         self.last_push = push
170         rho = clamp(L.rho0 - L.kI * xn.I_U - L.kE * xn.E, L.rho_min, L.rho0)
171         converged = push <= L.push_release
172         load_sized = o["load"] * self.current_cap if L.size_at_full_cap else o["load"]
173         n_req = math.ceil(n * load_sized / rho) + math.ceil(L.kq * o["q"] * n)
174         delta = max(-L.down_max, min(L.up_max, n_req - n))
175         if delta < 0 and not converged:
176             delta = 0
177         if delta > 0 and o["q"] < L.guard_queue and push < L.push_add:
178             delta = 0
179         if delta < 0 and (o["q"] >= L.guard_queue or n_req > n - L.down_band):
180             delta = 0
181         self.surplus_streak = self.surplus_streak + 1 if delta < 0 else 0
182         if delta < 0 and (self.surplus_streak < L.down_dwell or self.since_add < L.down_after_add):
183             delta = 0
184         ceiling = L.power_ceiling_backlog if o["q"] >= L.guard_queue else L.power_ceiling
185         if delta > 0 and (o["power"] >= ceiling or n >= L.max_nodes):
186             delta = 0
187         if o["security"] > 0.5:
188             delta = min(delta, 0)
189         self.since_add = 0 if delta > 0 else self.since_add + 1
190         if delta < 0:
191             self.surplus_streak = 0
192         at_envelope = o["power"] >= ceiling
193         if o["q"] < L.guard_queue and xn.U >= L.U_gate:
194             cap = clamp(self.current_cap * o["load"] * (1.0 + L.margin), L.cap_min, 1.0)
195         elif o["q"] >= L.guard_queue and L.spread_step > 0.0:
196             if at_envelope:
197                 cap = clamp(self.current_cap - L.spread_step, L.cap_min, 1.0)
198             else:
199                 cap = 1.0 if L.backlog_release else (self.current_cap if n < L.max_nodes else 1.0)
200         elif o["q"] >= L.guard_queue and L.backlog_release:
201             cap = 1.0
202         else:
203             cap = 1.0
204         cap = min(cap, clamp(1.0 - L.cap_gain * max(0.0, xn.B - L.B_cap), L.cap_min, 1.0))
205         if delta > 0:
206             cap = max(cap, self.current_cap)
207         command = clamp(0.5 * (v_eff(xn, self.p) / max(self.p.c, 1e-9) + 1.0), 0.0, 1.0)
208         d = {
209             "node_delta": int(delta),
210             "change_permitted": xn.U >= L.U_gate and o["security"] <= 0.5,
211             "power_cap": cap,
212             "rollback_authorized": xn.S >= L.S_rollback or xn.U < U_before or o["security"] > 0.5,
213             "route_shift": o["network"] > L.route_network,
214             "state": {"E": xn.E, "U": xn.U, "I_U": xn.I_U, "S": xn.S, "B": xn.B},
215             "demand": rho,
216             "load": o["load"],
217             "command": command,
218         }
219         self.log.append({"mode": self.mode, "killed": self.killed, **{k: v for k, v in d.items() if k != "state"},
220                        **d["state"]})
221     return d
222
223
224 def directive_to_actions(d: Dict[str, Any], stack_state: Dict[str, Any], proposals: List[Dict[str, Any]],
225                          cfg, direct_actuation: bool = False) -> List[Dict[str, Any]]:
226     """Directive -> executable stack actions. direct_actuation: capacity is owned by the governor; no Terraform plans."""
227
228     tag = "Omni-Compass"
229     out: List[Dict[str, Any]] = []
230     cur = int(stack_state["actual_nodes"])
231     target = int(max(cfg.minimum_nodes, min(cfg.maximum_nodes, cur + int(d["node_delta"]))))
232     if target != cur:
233         out.append({"manager": tag, "action": "nodes", "target": target, "direction": 1 if target > cur else -1})
234     if not direct_actuation and d["change_permitted"] and any(p["action"] == "terraform_plan" for p in proposals):
235         tf_dir = 1 if target > stack_state["terraform_desired_nodes"] else -1
236         live_dir = 1 if target > cur else -1 if target < cur else 0
237         if abs(target - stack_state["terraform_desired_nodes"]) >= 2 and live_dir in (0, tf_dir):
238             out.append({"manager": tag, "action": "terraform_plan", "target": target,
239                        "direction": 1 if target > stack_state["terraform_desired_nodes"] else -1})
240     reps = [int(p["target"]) for p in proposals if p["action"] == "replicas"]

```

```

240     if direct_actuation:
241         cur_r = int(stack_state["replicas"])
242         rt = max(20, min(500, math.ceil(cur_r * float(d["load"]) / max(float(d["demand"]), 1e-9))))
243         if (rt > cur_r and d["node_delta"] >= 0) or (rt < cur_r and d["node_delta"] < 0):
244             out.append({"manager": tag, "action": "replicas", "target": rt, "direction": 1 if rt > cur_r else -1})
245         reps = []
246     if reps:
247         cur_r = int(stack_state["replicas"])
248         rt = None
249         if d["node_delta"] >= 0 and max(reps) > cur_r:
250             rt = max(reps)
251         elif d["node_delta"] < 0 and min(reps) < cur_r:
252             rt = min(reps)
253         if rt is not None:
254             out.append({"manager": tag, "action": "replicas", "target": rt,
255                        "direction": 1 if rt > cur_r else -1})
256     cap = float(d["power_cap"])
257     if any(a["direction"] > 0 and a["action"] in ("nodes", "terraform_plan", "replicas") for a in out):
258         cap = max(cap, float(stack_state["power_cap"]))
259     if abs(cap - stack_state["power_cap"]) > 1e-9:
260         out.append({"manager": tag, "action": "power_cap", "target": cap,
261                    "direction": -1 if cap < stack_state["power_cap"] else 1})
262     if d["route_shift"]:
263         out.append({"manager": tag, "action": "route_shift", "target": 1.0, "direction": 1})
264     if any(p["action"] == "rollback" for p in proposals) and d["rollback_authorized"]:
265         out.append({"manager": tag, "action": "rollback", "target": 1.0, "direction": -1})
266     alerts = [p for p in proposals if p["action"] == "alert"]
267     if alerts:
268         out.append({"manager": tag, "action": "alert", "target": max(int(p["target"]) for p in alerts), "direction": 1})
269 )
270
271
272 THROUGHPUT_MODE = dict(backlog_release=True, power_ceiling=9.0, power_ceiling_backlog=9.0, spread_step=0.0, B_cap=9.0,
273                        size_at_full_cap=True, kI=0.05, kE=0.40, cap_min=0.65, down_band=1, down_dwell=10,
274                        down_after_add=3, kq=0.30, U_gate=0.45, push_release=0.01)
275
276
277 FLEET_MODE = dict(THROUGHPUT_MODE, rho=0.829, rho_min=0.769, kI=0.072, kE=0.487, kq=0.635, down_band=3, down_dwell=1,
278                  down_after_add=5, push_release=0.053, cap_min=0.695, margin=0.131, U_gate=0.85, up_max=8)
279 FLEET_BALANCED_MODE = dict(FLEET_MODE, push_release=0.0098, push_add=0.0098)
280 FLEET_WEAR_MODE = dict(FLEET_MODE, push_release=0.0035, push_add=0.0035)
281 FLEET_DECISION_SECONDS = 60
282
283 def mode_law(mode: str, base: "AllocationLaw" = None) -> "AllocationLaw":
284     """Operating modes.
285
286     power_protect    the site power limit is enforced (envelope, spread-and-throttle, bath cap; shield I4 active)
287     fleet            throughput mode with constants selected on the 15-second fleet harness (fleet/), decisions every 60
288 s;
289     fleet_balanced   energy-first: the equation (2) release gate is lenient and the add gate is off
290     fleet_wear        fleet with the equation (2) gate active in both directions at 0.0098 (release and add)
291     throughput        fleet with the equation (2) gate active in both directions at 0.0035 (release and add)
292                     the power envelope is not enforced (same power rules as the native stack); caps released on backlog;
293                     capacity sized at full power (load * cap_now), so power trimming does not trigger machine starts;
294                     constants selected by constrained search on development seeds (minimum energy subject to backlog,
295                     health, recovery, reversal and invariant constraints)
296
297     """
298     from dataclasses import replace
299     base = base or AllocationLaw()
300     if mode == "power_protect":
301         return base
302     if mode == "throughput":
303         return replace(base, **THROUGHPUT_MODE)
304     if mode == "fleet":
305         return replace(base, **FLEET_MODE)
306     if mode == "fleet_balanced":
307         return replace(base, **FLEET_BALANCED_MODE)
308     if mode == "fleet_wear":
309         return replace(base, **FLEET_WEAR_MODE)
310     raise ValueError(mode)

```

```

==== omnicompass/authority_contract.py (57 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Executable authority/evidence contracts for Omni-Compass.
5
6  This module does not claim that a plant obeyed Omni. It defines the minimum links that must
7  exist before software may promote a claim from COMPUTED -> REQUESTED -> ENFORCED -> ATTRIBUTED
8  -> BENEFICIAL -> RESTORED. Missing evidence can only lower a claim level, never raise it.
9  """
10 from __future__ import annotations
11 from dataclasses import dataclass, asdict
12 from enum import IntEnum
13 from typing import Any, Iterable
14
15 class ClaimLevel(IntEnum):
16     COMPUTED=0; REQUESTED=1; ENFORCED=2; ATTRIBUTED=3; BENEFICIAL=4; RESTORED=5
17
18 @dataclass(frozen=True)
19 class AuthorityContract:
20     muscle_id: str
21     resource: str
22     writer: str
23     command: str
24     bounds: str
25     readback: str
26     conflict_set: tuple[str,...]
27     meter: tuple[str,...]
28     guardrails: tuple[str,...]
29     restore: str
30     evidence_class: str
31     def to_dict(self):
32         d=asdict(self); d['conflict_set']=list(self.conflict_set); d['meter']=list(self.meter); d['guardrails']=list(se
lf.guardrails); return d
33
34 GPU_POWER = AuthorityContract(
35     'nvidia.gpu.power_limit','GPU board power envelope','omni_controller/gpu_governor.py',
36     'nvidia-smi -i <gpu> -pl <watts>','device min <= watts <= frozen start limit',
37     'power.limit + enforced.power.limit',
38     ('VBIOS','BMC/SMBPBI/chassis','application clocks','foreign host writer'),
39     ('DCGM/NVML device joules','integrated board power','optional wall meter','RAPL package meter (separate domain)'),
40     ('successful work','SLO','no thermal/HwPowerBrake confound','Watch writes == 0'),
41     'restore frozen starting board-power limit and read back','physical')
42
43 RAPL_METER = AuthorityContract(
44     'intel.rapl.meter','CPU package energy meter','read-only',
45     'read energy_uj','read-only','start/end energy_uj + max_energy_range_uj',(),
46     ('RAPL package joules','optional DRAM joules','optional psys joules'),
47     ('wrap-aware delta','domain labels remain separate'), 'none (read-only)','physical-meter')
48
49 K8S_REPLICA = AuthorityContract(
50     'kubernetes.replicas','workload replica allocation','Omni Kubernetes adapter/native controller',
51     'scale/patch replica target','declared min/max + safety envelope','observed desired/actual replicas',
52     ('HPA','operator','GitOps/deployment writer'),('work','latency','pending/backlog','optional node/wall energy'),
53     ('SLO','health','backlog','unsafe == 0'),'restore/hand back original controller state','external-software')
54
55 CONTRACTS={x.muscle_id:x for x in (GPU_POWER,RAPL_METER,K8S_REPLICA)}
56
57 def catalogue(): return {k:v.to_dict() for k,v in CONTRACTS.items()}

```

```

==== omnicompass/bowl.py (174 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """The bowl: one smooth law for every muscle, and the plug every muscle is wired through.
5
6  A muscle is a plug with one wire in and one wire out. The plug reads its meters and writes its lever; all the
7  thinking is here, in the brain, written once for every muscle.
8
9  The plug (Plug)
10 read     the service reading (response time, temperature, queue) and the lever as it stands
11 write    one value; the plug clips it to the cover, sends it, and reads back what the device took
12 restore  the lever back to the snapshot, the value read once before the first write; it never moves after that
13 one writer if the lever is found at a value the brain did not write, someone else owns it: the brain stops writing
14           and leaves that value alone (restoring over it would fight the new owner)
15
16 Two bands
17 cover    the outer band, the lever's hard range (the device's or the buyer's lowest and highest setting); every
18           write is clipped to it, so nothing the math does can set the lever outside
19 bowl     the service reading as a position in its band, 0 to 1 (0: calm, 1: the service line); the brain pulls it
20           to the bottom of the bowl, the center (0.5 by default); the walls run to 0.05 and 0.95, and the last 5% on
21           each side is cushion (10% in all) before the line
22
23 The force (Bowl.force), the physics of a ball in a bowl with the right friction
24 pull     K_P x (position - center): gentle near the bottom, harder up the walls
25 push     K_D x velocity: whatever is shoving the position (a load rising, heat building) is met by an equal and
26           opposite push; it is also the friction that stops the ball sloshing past the bottom (with K_D chosen for
27           critical damping it glides to the center and stops, no overshoot, no ringing)
28 smooth   the force is A x tanh(raw / A): a spring near the center that bends over and flattens into its maximum A,
29           never a corner, never a hammer; the same shape the engine already uses for v_eff (omnicompass/core.py)
30 fail up  past the 0.95 wall the up side goes to its full force at once, and the down side may not act until the
31           position is back inside the bowl
32
33 Two forces (antagonist pairs)
34 up       a positive force adds capacity, power, cooling or speed (scale out, raise a clock, start a chiller)
35 down     a negative force takes it back (scale in, lower a limit, warm a setpoint)
36 each side has its own gain (lever units per unit of force), because adding and taking back do not cost the same;
37 a muscle with two wires (a GPU's clock range and power limit) gives each side its own lever; a one-wire muscle
38 carries both directions on its one lever
39
40 The frozen engine (omnicompass/core.py) and its proofs are untouched; this is a separate law, run as its own arm.
41 """
42 from __future__ import annotations
43
44 import math
45 from dataclasses import dataclass, field
46 from typing import Optional
47
48
49 def clamp(v, lo, hi):
50     return lo if v < lo else hi if v > hi else v
51
52
53 @dataclass
54 class Band:
55     """A reading's band: lo is calm (position 0), hi is the line (position 1)."""
56     lo: float
57     hi: float
58     center: float = 0.5
59     cushion: float = 0.05
60
61     def position(self, x: float) -> float:
62         return (x - self.lo) / (self.hi - self.lo)
63
64     @property
65     def wall_low(self) -> float:
66         return self.cushion
67
68     @property
69     def wall_high(self) -> float:
70         return 1.0 - self.cushion
71
72
73 @dataclass
74 class Bowl:
75     """The brain's law for one muscle. dt: seconds between decisions; tau: the muscle's response time in seconds
76     (how long the service reading takes to follow the lever); kd defaults to critical damping for that response."""
77     band: Band
78     dt: float
79     tau: float
80     kp: float = 1.0

```

```

81     kd: Optional[float] = None
82     authority: float = 1.0
83     smooth: float = 0.5          # the velocity estimate's smoothing (0: raw difference, near 1: heavily smoothed)
84     p: Optional[float] = field(default=None, init=False)
85     v: float = field(default=0.0, init=False)
86
87     def __post_init__(self):
88         if self.kd is None:
89             # a first-order muscle with response tau under a P-D pull: critical damping when kd = 2 sqrt(kp tau) - 1
90             # scaled to the decision period; never negative
91             self.kd = max(0.0, 2.0 * math.sqrt(self.kp * self.tau / self.dt) - 1.0) * self.dt
92
93     def sense(self, reading: float) -> float:
94         p = self.band.position(reading)
95         if self.p is not None:
96             self.v = self.smooth * self.v + (1.0 - self.smooth) * (p - self.p) / self.dt
97         self.p = p
98         return p
99
100    def force(self, reading: float) -> float:
101        """Positive: the up side (add capacity); negative: the down side (take it back). Bounded by the authority."""
102        p = self.sense(reading)
103        if p >= self.band.wall_high:
104            return self.authority # fail up: past the wall, full force up at once
105        raw = self.kp * (p - self.band.center) + self.kd * self.v
106        f = self.authority * math.tanh(raw / self.authority)
107        if f < 0.0 and self.v > 0.0 and p > self.band.center:
108            f = 0.0 # never take back while it climbs above the center
109        return f
110
111
112    class ForeignWriter(Exception):
113        pass
114
115
116    class Plug:
117        """One wire in, one wire out. Subclasses implement _read_service, _read_lever and _send; everything else is here."""
118
119    def __init__(self, lo: float, hi: float, tolerance: float = 1e-6):
120        self.lo, self.hi, self.tol = lo, hi, tolerance
121        self.snapshot: Optional[float] = None
122        self.last_written: Optional[float] = None
123        self.owned = True
124        self.writes = 0
125        self.clipped = 0
126
127    # the wires -----
128    def _read_service(self) -> float:
129        raise NotImplementedError
130
131    def _read_lever(self) -> float:
132        raise NotImplementedError
133
134    def _send(self, value: float) -> None:
135        raise NotImplementedError
136
137    # the contract -----
138    def attach(self) -> float:
139        """Read the lever once, before any write: the restore point. It never moves after this."""
140        self.snapshot = self._read_lever()
141        return self.snapshot
142
143    def read(self) -> float:
144        return self._read_service()
145
146    def lever(self) -> float:
147        return self._read_lever()
148
149    def write(self, value: float) -> Optional[float]:
150        """Clip to the cover, check nobody else moved the lever, send, read back. None: not written (not owned)."""
151        if not self.owned:
152            return None
153        now = self._read_lever()
154        expected = self.last_written if self.last_written is not None else self.snapshot
155        if expected is not None and abs(now - expected) > self.tol:
156            self.owned = False # someone else moved it: it is theirs now
157            raise ForeignWriter(f"lever at {now}, last written {expected}")
158        v = clamp(value, self.lo, self.hi)
159        self.clipped += int(v != value)
160        if abs(v - now) <= self.tol:

```

```
161         return now
162         self._send(v)
163         self.last_written = self._read_lever()
164         self.writes += 1
165         return self.last_written
166
167     def restore(self) -> bool:
168         """The lever back to the snapshot. Left alone when someone else owns it. True: at the snapshot (or not ours)."""
169
170         if not self.owned or self.snapshot is None:
171             return True
172         if abs(self._read_lever() - self.snapshot) > self.tol:
173             self._send(self.snapshot)
174             self.last_written = None
175             return abs(self._read_lever() - self.snapshot) <= self.tol
```

```

==== omnicompass/ce_score.py (30 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Control-effort index. Experiment-defined. Sweep weights; don't trophy one set."""
5  from __future__ import annotations
6  from typing import Dict
7
8  SWEEPS = {
9      "balanced": dict(replica_change=1, pod_move=1, node_provision=8, node_suspend=8, resize=2, reversal=4, power_cap_travel=1),
10     "node_heavy": dict(replica_change=1, pod_move=1, node_provision=16, node_suspend=16, resize=2, reversal=4, power_cap_travel=1),
11     "replica_heavy": dict(replica_change=4, pod_move=2, node_provision=8, node_suspend=8, resize=2, reversal=8, power_cap_travel=1),
12     "resize_heavy": dict(replica_change=1, pod_move=1, node_provision=8, node_suspend=8, resize=8, reversal=4, power_cap_travel=1),
13     "uniform": dict(replica_change=1, pod_move=1, node_provision=1, node_suspend=1, resize=1, reversal=1, power_cap_travel=1),
14 }
15
16
17 def ce(row: Dict, weights: Dict[str, float]) -> float:
18     return (
19         weights.get("replica_change", 0) * float(row.get("replica_change_units", 0))
20         + weights.get("pod_move", 0) * float(row.get("evictions", 0))
21         + weights.get("node_provision", 0) * float(row.get("node_starts", 0))
22         + weights.get("node_suspend", 0) * float(row.get("node_stops", 0))
23         + weights.get("resize", 0) * float(row.get("request_changes", 0))
24         + weights.get("reversal", 0) * float(row.get("replica_reversals", 0))
25         + weights.get("power_cap_travel", 0) * float(row.get("power_cap_travel", 0))
26     )
27
28
29 def all_sweeps(row: Dict) -> Dict[str, float]:
30     return {name: ce(row, w) for name, w in SWEEPS.items()}

```

```

==== omnicompass/closure.py (250 lines) ====
1  """Closure governor: machines governed by the Omni-Compass closure law, preemptive coherence and the dual-bath tracker.
2
3  Sources (the owner's manuscript): Section 5 (5c preemptive coherence: forward projection  $\tau_{\text{fwd}} = \Phi_{\text{dt}}(x)$  and
4  correction  $-\lambda \text{grad } \psi$ ), Chapter 20 (master closure law  $F = G_0 + G_c$ ,  $G_c = -\rho \beta D_h$ , zero in the interior,
5  inward near the boundary with margin  $\delta$ ), Chapters 29-30 (the wheel: expansion, turning point  $dE/dt = 0$ , compression
6  ;
7  release only after the turn, ignition before the threshold), Chapter 31 (dual-bath exchange  $dU/dt = -\alpha E + \beta S$ ,
8   $dE/dt = \alpha U - \gamma B$ ).
9
10 Cluster realisation (one pool)
11 state  $r(t)$  requested cores (what the pods ask for);  $n$  machines;  $c = \text{cores} \times \text{ALLOC}$  per machine
12 dual bath level  $L$  and rate  $v$  of  $r$ , the exchange pair of Chapter 31 in discrete form:
13 innovation  $e = r - (L + v \text{ dt})$ 
14  $L \leftarrow L + v \text{ dt} + a e$  (coherence bath absorbs the observation)
15  $v \leftarrow (1 - g) v + b e / \text{dt}$  (deviation bath exchanges with it;  $g$  = redistribution damping)
16 projection  $r_{\text{fwd}}(\tau) = L + v \tau$  for  $\tau$  in  $[0, H]$  (5c:  $\Phi$  over the look-ahead horizon)
17 admissible  $h(\tau) = r_{\text{fwd}}(\tau) / (n c) - \rho_{\text{max}} \leq 0$  (Chapter 20: the admissible domain)
18 closure  $G_c$  if  $\max\{\tau \leq H_{\text{add}}\} h(\tau) > -\delta$  (projected boundary approach, within one boot of lead):
19 add  $k = \text{ceil}((\max r_{\text{fwd}} / (\rho_{\text{max}} - \delta) - n c) / c)$  machines (beta sized to restore  $\delta$ )
20 )
21 interior  $G_0$  energy descent: remove one machine only if it stays admissible with margin for the whole release horizon
22 ,
23 max $\{\tau \leq H_{\text{rel}}\} r_{\text{fwd}}(\tau) / ((n - 1) c) \leq \rho_{\text{max}} - \delta$ ,
24 and only after the turn ( $v \leq 0$ : past the peak, the metric flip), and only while the six-state engine's
25 equation (2) push  $\leq \text{push\_release}$ 
26 interior nothing else happens: inside the domain the platform physics runs unmodified ( $G_c = 0$ )
27 """
28 from __future__ import annotations
29
30 import math
31 from dataclasses import dataclass
32
33 @dataclass(frozen=True)
34 class ClosureLaw:
35     rho_max: float = 0.95 # admissibility boundary: requested cores / allocatable cores
36     delta: float = 0.05 # inwardness margin
37     H_add: int = 8 # look-ahead for the closure correction (ticks);  $\geq$  the boot delay
38     H_rel: int = 40 # look-ahead a release must survive (ticks)
39     a: float = 0.5 # dual-bath coupling: level absorption
40     b: float = 0.1 # dual-bath coupling: rate exchange
41     g: float = 0.05 # redistribution damping of the rate bath
42     push_release: float = 0.2 # engine gate on release
43     turn: bool = True # release only after the turning point ( $v \leq 0$ )
44     delta_rel: float = -1.0 # release band: a release must fit at  $\rho_{\text{max}} - \delta_{\text{rel}}$  (hysteresis;  $< 0$  means  $= \delta$ )
45
46     z: float = 0.0 # deviation-bath band: a release must survive  $z$  standard deviations of the tracker's
47     # innovation over the release horizon (Chapter 31: the deviation bath carries the noise;
48     # the band is wide where demand is noisy, tight where it is calm)
49     tone: bool = False # muscle tone: release = park (alive, low power, instant wake), not power-off; parked
50     # machines beyond the reserve needed within  $\text{tone}_H$  are powered off
51     site: bool = False # whole body (multi-cluster sites): the law runs on the site total, one warm reserve for
52     # the site, and traffic shift lets a cluster use another's already-powered machines
53     tone_H: int = 960 # reserve horizon (ticks): the largest demand seen over this window sets the warm reserv
54
55     e
56     coord: bool = False # nervous-system coordination: pods decided first, machines on those pods, and the two
57     # organs never move in opposite directions in one decision
58     wake_first: bool = False # whole body: a site that needs a machine wakes a parked one in any cluster before it co
59
60     ld-
61     dwell: int = 0 # boots one (traffic shift carries the load; a wake costs no start, a boot costs one)
62     # release only after this many consecutive calm decisions (resource-aware envelope,
63     # Proposition 2: no release outside the calm set;  $0$  = no dwell requirement)
64     turn_rise: float = 0.0 # the turning-point veto binds only for a rise larger than this share of one machine ove
65
66     r
67     # the release horizon ( $v * H_{\text{rel}} > \text{turn\_rise} * c$ );  $0$  = any rise vetoes (the benchmarked
68     # law). A rise smaller than the release band cannot carry demand through the band, so it
69     # is no reason to hold a machine (one small rise must not hold every machine for a run)
70
71     class ClosureNodes:
72         def __init__(self, law: ClosureLaw = ClosureLaw()):
73             self.L = None; self.v = 0.0; self.law = law; self.calm = 0; self.s2 = 0.0; self.hist = []
74
75         def observe(self, r: float, dt: float = 1.0) -> None:
76             a, b, g = self.law.a, self.law.b, self.law.g
77             if self.law.tone:
78                 self.hist.append(r)
79                 if len(self.hist) > self.law.tone_H:
80                     self.hist.pop(0)
81             if self.L is None:

```

```

74         self.L = r; return
75         pred = self.L + self.v * dt
76         e = r - pred
77         self.s2 = 0.95 * self.s2 + 0.05 * e * e
78         self.L = pred + a * e
79         self.v = (1.0 - g) * self.v + b * e / dt
80
81     def reserve(self, c: float) -> int:
82         """Machines the law expects to need within the tone horizon (largest recent demand at the working boundary)."""
83         if not self.hist:
84             return 0
85         return int(math.ceil(max(self.hist) / ((self.law.rho_max - self.law.delta) * c) - 1e-9))
86
87     def fwd_max(self, H: int) -> float:
88         return max(self.L + self.v * tau for tau in range(0, H + 1)) if self.L is not None else 0.0
89
90     def decide(self, n: int, c: float, push: float, n_min: int, n_max: int) -> int:
91         """Target machine count for this tick."""
92         L = self.law
93         if self.L is None:
94             return n
95         peak_add = max(self.fwd_max(L.H_add), 0.0)
96         if n <= 0 or peak_add / (n * c) - L.rho_max > -L.delta:
97             self.calm = 0
98             need = math.ceil(peak_add / ((L.rho_max - L.delta) * c) - 1e-9)
99             return int(min(n_max, max(n_min, need, n)))
100         peak_rel = max(self.fwd_max(L.H_rel), 0.0) + L.z * math.sqrt(self.s2 * max(1, L.H_rel)) * L.a
101         band = L.delta if L.L_delta_rel < 0 else L.L_delta_rel
102         calm = n - 1 >= n_min and peak_rel / ((n - 1) * c) <= L.rho_max - band \
103             and (not L.turn or self.v * L.H_rel <= L.turn_rise * c) and push <= L.push_release
104         self.calm = self.calm + 1 if calm else 0
105         if calm and self.calm > L.dwell:
106             self.calm = 0
107         return n - 1
108     return n
109
110
111 # -----
112 # Self-calibrating closure law: every margin is derived, none is tuned per workload.
113 #
114 # tracker          dual bath (level L, rate v); the gains are chosen by the data: a bank of trackers from slow-trend to f
115 # ast-             trend (a, b) runs side by side and the one with the smallest running one-step prediction error drives
116 # the              the
117 #                  forecast (model selection by prediction error; the fastest member is the critically damped a = 1/2,
118 #                  b = a^2 / (2 - a))
119 # noise            sigma^2 = running variance of the tracker's one-step innovation (the deviation bath of Chapter 31)
120 # add horizon      H_add = boot delay + 1 tick: a machine ordered now is serving when the projection arrives
121 # release hor.     H_rel = boot delay / park fraction: the energy break-even between keeping a machine parked and booting
122 #                  it cold again (a parked machine draws park_frac x idle per tick, a cold start draws boot x idle)
123 # boundary         requested cores <= allocatable cores (rho_max = 1, the physical limit); no fixed margin
124 # margin           z(t) sigma sqrt(H): the forecast error over the horizon at confidence z;
125 #                  z(t) = z95 x max(1, S / S*), S the six-state engine's stress (equation 6) and S* its equilibrium,
126 #                  delta - alpha_s S* - (3/4) beta_s S^*^2 = 0: the engine widens the margin when it is stressed
127 # Gc (add)         if projected peak over H_add + margin > n c: add ceil((peak + margin) / c) - n machines
128 # G0 (release)     one machine, only if projected peak over H_rel + margin over max(H_rel, T*) <= (n - 1) c (the machine
129 #                  is
130 #                  not expected back within the learned break-even T* below), after the turn (v <= 0), and while
131 #                  the engine's push <= push_release; the released machine is parked
132 # power-off        learned break-even (ski rental with measured return times): every release at level k opens a record
133 #                  that closes when demand needs k machines again; X = the time it took. A parked machine is powered off
134 #                  when it has been parked T* ticks, T* = argmin_T sum_i cost_i(T), cost_i = park_frac X_i if X_i <= T,
135 #                  else park_frac T + boot (energy in idle-machine ticks; still-open records are right-censored at their
136 #                  age). Before any record exists T* = boot / park_frac, the deterministic ski-rental rule (never more th
137 #                  an
138 #                  twice the energy of a controller that knows the future). Nothing is tuned: T* is computed from the
139 #                  workload's own return times, per workload, while it runs
140 # -----
141 @dataclass(frozen=True)
142 class AutoClosureLaw:
143     boot: int = 6 # boot delay of a cold machine (ticks): a property of the plant
144     park_frac: float = 0.25 # parked power / idle power: a property of the hardware
145     z95: float = 1.6449 # one-sided 95% normal quantile
146     a: float = 0.5
147     push_release: float = 0.2
148     site: bool = False
149     # read by the plant (same interface as ClosureLaw)
150     tone: bool = True
151     rho_max: float = 1.0
152     delta: float = 0.0

```

```

151     @property
152     def b(self) -> float:
153         return self.a * self.a / (2.0 - self.a)
154
155     @property
156     def H_add(self) -> int:
157         return self.boot + 1
158
159     @property
160     def H_rel(self) -> int:
161         return int(math.ceil(self.boot / self.park_frac))
162
163     @property
164     def tone_H(self) -> int:
165         return self.H_rel
166
167
168 def stress_equilibrium(delta: float, alpha_s: float, beta_s: float) -> float:
169     """Positive root of  $\delta - \alpha_s S - (3/4) \beta_s S^2 = 0$  (the engine's stress equilibrium, equation 6)."""
170     qa = 0.75 * beta_s
171     return (-alpha_s + math.sqrt(alpha_s * alpha_s + 4.0 * qa * delta)) / (2.0 * qa) if qa > 0 else delta / alpha_s
172
173
174 class AutoClosureNodes:
175     def __init__(self, law: AutoClosureLaw = AutoClosureLaw(), s_eq: float = None):
176         self.law = law; self.L = None; self.v = 0.0; self.s2 = 0.0; self.n_obs = 0
177         self.S = 0.0; self.s_eq = s_eq if s_eq else stress_equilibrium(0.5, 0.12, 0.10)
178         self.hist = []; self.ages = []; self.n_last = 0
179         self.t = 0; self.open = []; self.closed = [] # release records: (level, t0) open; X closed
180
181     BANK = ((0.2, 0.002), (0.2, 0.02), (0.5, 0.005), (0.5, 0.05), (0.5, 1.0 / 6.0))
182
183     def observe(self, r: float, dt: float = 1.0) -> None:
184         if self.L is None:
185             self.L = r; self.bank = [[r, 0.0, 0.0] for _ in self.BANK]; return
186         self.n_obs += 1; self.t += 1
187         self.ages = [x + 1 for x in self.ages]
188         w = max(1.0 / self.n_obs, 0.02) # running mean first, then a 50-tick exponential window
189         for (a, b), m in zip(self.BANK, self.bank):
190             pred = m[0] + m[1] * dt
191             e = r - pred
192             m[2] = (1.0 - w) * m[2] + w * e * e
193             m[0] = pred + a * e
194             m[1] = m[1] + b * e / dt
195         best = min(self.bank, key=lambda m: m[2])
196         self.L, self.v, self.s2 = best
197
198     def z(self) -> float:
199         return self.law.z95 * max(1.0, self.S / self.s_eq) if self.s_eq > 0 else self.law.z95
200
201     def peak(self, H: int, H_noise: int = None) -> float:
202         """Projected peak over H ticks (level + trend) plus the forecast-error margin accumulated over H_noise ticks."""
203
204         if self.L is None:
205             return 0.0
206         Hn = H if H_noise is None else H_noise
207         return max(self.L + self.v * tau for tau in range(0, H + 1)) + self.z() * math.sqrt(self.s2 * Hn)
208
209     GRID = (0, 6, 12, 24, 48, 96, 192, 384, 768, 1440)
210
211     def threshold(self) -> int:
212         L = self.law
213         if not self.closed and not self.open:
214             return L.H_rel
215         cens = [self.t - t0 for _, t0 in self.open]
216         def cost(T):
217             c = sum(L.park_frac * x if x <= T else L.park_frac * T + L.boot for x in self.closed)
218             return c + sum(L.park_frac * T + L.boot if a > T else L.park_frac * a for a in cens)
219         return min(self.GRID, key=lambda T: (cost(T), T))
220
221     def reserve(self, c: float) -> int:
222         """Machines to keep powered (running + parked younger than the learned break-even T*); older parked go off."""
223         T = self.threshold()
224         self.ages = [x for x in self.ages if x < T]
225         return self.n_last + len(self.ages)
226
227     def decide(self, n: int, c: float, push: float, n_min: int, n_max: int) -> int:
228         L = self.law
229         if self.L is None:
230             return n
231         need = int(math.ceil(max(self.peak(L.H_add), 0.0) / c - 1e-9))

```

```

231         still = []
232         for lvl, t0 in self.open:
233             if need >= lvl:
234                 self.closed.append(self.t - t0)
235             else:
236                 still.append((lvl, t0))
237         self.open = still[-200:]; self.closed = self.closed[-400:]
238         if n <= 0 or need > n:
239             t = int(min(n_max, max(n_min, need)))
240             self.ages.sort(); del self.ages[max(0, len(self.ages) - (t - n)):]
241             self.n_last = t
242             return t
243         # release only if the machine is not expected back within the learned break-even T*: the trend is projected over
244         r
245         # the release horizon, the forecast error is accumulated over max(H_rel, T*)
246         if n - 1 >= n_min and self.peak(L.H_rel, max(L.H_rel, self.threshold())) <= (n - 1) * c and self.v <= 0.0 \
247             and push <= L.push_release:
248             self.ages.append(0); self.n_last = n - 1; self.open.append((n, self.t))
249             return n - 1
250         self.n_last = n
251         return n

```

```

==== omnicompass/compass.py (163 lines) ====
1  """The Omni-Compass, computed.
2
3  I read my own state on the compass every decision and say where I am, in the compass's own terms.
4
5  The wheel (Chapter 29). My deviation E, scaled by its ceiling E_max, runs along the horizontal; its rate of change runs
6  along the vertical. A reading is a point on that wheel. Its heading is measured clockwise from north, as on the face:
7
8      +   A       0 deg   rising through rest
9      ■   Δ       45 deg   expansion: above rest and rising, exchange under way
10     >   B       90 deg   peak extension: the turn, where the metric flips (k -> -k)
11     ■   Λ       135 deg  dispersion: above rest and easing, the ceiling holds
12     -   Ω       180 deg  falling through rest
13     ≈   Π       225 deg  compression: below rest and settling into the basin
14     <   Γ       270 deg  deepest compression: the turn at the floor
15     ♦   Ψ       315 deg  re-alignment: below rest and rising, ignition ahead
16
17  The four quadrants are the strokes of the closed circle (Chapters 29-30):
18      I   Expansion    above rest, rising
19      II  Dispersion   above rest, easing
20      III Compression below rest, falling
21      IV  Re-alignment below rest, rising
22  The wheel turns I -> II -> III -> IV -> I. Crossing from IV into I is ignition; the turn at peak extension is the
23  metric flip. The state is continuous through every crossing: nothing switches off, nothing restarts.
24
25  The rim carries the 24 letters in their places on the face, 15 degrees each, clockwise from A at north.
26
27  The axle is the structural basin S. Its rest point S* solves equation (6):  $\delta - \alpha_s S - (3/4) \beta_s S^2 = 0$ .
28
29  Closing the circle (Chapter 1, the Unified Circle Principle):
30      X' = G(X),   X(0) in Omega           Omega is the living band: every level inside [0.05, 0.95]
31      G(X) . n(X) <= 0 on the boundary    at a boundary the next move points inward, never outward
32      grad L(X) . G(X) <= 0              the ledger L, the engine's composite storage over all six states
33                                         (omnicompass.storage.composite_V), descends when nothing is forcing it
34      => lim X(t) in M*                    the state settles into the basin
35  I check all three every decision and report them. The ledger may rise while outside load forces me; I report that as
36  forcing, never hide it.
37  """
38  from __future__ import annotations
39
40  import math
41  from dataclasses import dataclass, field
42  from typing import Dict, List, Optional
43
44  from omnicompass.nervous_system import BAND
45
46  RIM = ["A", "E", "Z", "Δ", "H", "Θ", "B", "I", "K", "Λ", "M", "N",
47        "Ω", "Ξ", "O", "Π", "P", "Σ", "Γ", "T", "Y", "Ψ", "Φ", "X"]
48  POINTS = [{"+": "rising through rest"}, {"■": "expansion, exchange under way"}, {">": "peak extension, the turn"},
49            {"■": "dispersion, the ceiling holds"}, {"-": "falling through rest"}, {"≈": "compression, settling into the
50            basin"}, {"<": "deepest compression, the turn at the floor"}, {"♦": "re-alignment, ignition ahead"}]
51  QUADRANTS = {1: "Expansion", 2: "Dispersion", 3: "Compression", 4: "Re-alignment"}
52
53
54  def phi(S: float, alpha_s: float, beta_s: float, delta: float) -> float:
55      """Basin potential, equation (5)."""
56      return alpha_s * S * S / 2.0 + beta_s * S ** 3 / 4.0 - delta * S
57
58
59  def axle(alpha_s: float, beta_s: float, delta: float) -> float:
60      """S*, the rest point of the axle: the positive root of  $\delta - \alpha_s S - (3/4) \beta_s S^2 = 0$ ."""
61      q = 0.75 * beta_s
62      if q <= 0:
63          return delta / alpha_s if alpha_s > 0 else 0.0
64      return (-alpha_s + math.sqrt(alpha_s * alpha_s + 4.0 * q * delta)) / (2.0 * q)
65
66
67  def heading(e: float, rate: float) -> float:
68      """Degrees clockwise from north: north is rising (rate > 0), east is extension (e > 0)."""
69      if abs(e) < 1e-12 and abs(rate) < 1e-12:
70          return 0.0
71      return math.degrees(math.atan2(e, rate)) % 360.0
72
73
74  def quadrant(e: float, rate: float) -> int:
75      if e > 0:
76          return 1 if rate > 0 else 2
77      return 3 if rate < 0 else 4
78
79

```

```

80 def in_omega(levels: Dict[str, float]) -> Dict[str, bool]:
81     lo, hi = BAND
82     return {k: lo - 1e-9 <= v <= hi + 1e-9 for k, v in levels.items()}
83
84
85 def inward(level: float, move: float, eps: float = 0.01) -> bool:
86     """G . n <= 0 on the boundary of Omega: at (or past) the floor the move is not downward, at the ceiling not upward.
87     """
88     lo, hi = BAND
89     if level >= hi - eps and move > 0:
90         return False
91     if level <= lo + eps and move < 0:
92         return False
93     return True
94
95 @dataclass
96 class Compass:
97     """Reads the engine each decision and keeps the previous reading to form rates and the ledger step."""
98     E_max: float = 1.0
99     alpha_s: float = 0.12
100    beta_s: float = 0.10
101    delta: float = 0.5
102    prev_e: Optional[float] = None
103    prev_L: Optional[float] = None
104    scale: float = 0.0 # running RMS of the rate, so the wheel is round in its own units
105    turns: int = 0 # full circles closed (IV -> I crossings)
106    last_q: Optional[int] = None
107    trail: List[str] = field(default_factory=list)
108
109    def read(self, E: float, S: float, levels: Dict[str, float] = None, moves: Dict[str, float] = None,
110            forced: bool = False, x=None, p=None) -> Dict:
111        """x, p: the engine's full state and parameters; with them the ledger is the composite storage of all six
112        states. Without them it is the storage of the two the reading names, E^2/2 + Phi(S) - Phi(S*)."""
113        e = E / max(self.E_max, 1e-12)
114        raw = 0.0 if self.prev_e is None else e - self.prev_e
115        self.scale = raw * raw if self.scale == 0.0 else 0.9 * self.scale + 0.1 * raw * raw
116        rate = raw / math.sqrt(self.scale) if self.scale > 1e-18 else 0.0
117        h = heading(e, rate)
118        q = quadrant(e, rate)
119        if self.last_q == 4 and q == 1:
120            self.turns += 1
121        pt, meaning = POINTS[int(((h + 22.5) % 360) // 45)]
122        letter = RIM[int(((h + 7.5) % 360) // 15)]
123        s_star = axle(self.alpha_s, self.beta_s, self.delta)
124        parts = {}
125        if x is not None and p is not None:
126            from omnicompass.storage import composite_V
127            cv = composite_V(x, p, 1)
128            L = cv["V"]; parts = {k: round(v, 5) for k, v in cv.items() if k.startswith("V_")}
129            s_star = cv["S_star"]
130        else:
131            L = 0.5 * e * e + phi(S, self.alpha_s, self.beta_s, self.delta) - phi(s_star, self.alpha_s, self.beta_s, se
132        lf.delta)
133        dL = None if self.prev_L is None else L - self.prev_L
134        levels = levels or {}; moves = moves or {}
135        omega = in_omega(levels)
136        lo, hi = BAND
137        outside = {k: ("below the floor" if levels[k] < lo else "above the ceiling") for k, ok in omega.items() if not
138        ok}
139        heading_in = {k: (moves.get(k, 0.0) > 0 if levels[k] < lo else moves.get(k, 0.0) < 0) for k in outside}
140        inw = {k: inward(levels[k], moves.get(k, 0.0)) for k in levels}
141        r = {"heading_deg": round(h, 1), "letter": letter, "point": pt, "meaning": meaning,
142            "quadrant": q, "stroke": QUADRANTS[q], "e": round(e, 4), "rate": round(rate, 3),
143            "axle_S": round(S, 4), "axle_rest": round(s_star, 4), "ledger": round(L, 5),
144            "ledger_step": None if dL is None else round(dL, 5), "ledger_parts": parts,
145            "descent": None if dL is None else (dL <= 1e-9 or forced), "forced": forced,
146            "omega_held": all(omega.values()) if omega else True, "inward": all(inw.values()) if inw else True,
147            "outside": outside, "returning": heading_in,
148            "circles_closed": self.turns}
149        self.prev_e, self.prev_L, self.last_q = e, L, q
150        return r
151
152 def omega_words(r: Dict) -> str:
153     if r["omega_held"]:
154         return "Ω held"
155     parts = [{"k": w, "returning": r["returning"].get(k) else 'held'} for k, w in r["outside"].items()]
156     return "Ω: " + "; ".join(parts)
157

```

```
158 def say(r: Dict) -> str:
159     """One line in the compass's own terms."""
160     step = "" if r["ledger_step"] is None else f" {r['ledger_step']:+.4f}{' (forced)' if r['forced'] and r['ledger_step']
    ']' > 0 else ''}"
161     return (f"{r['letter']} {r['point']} {r['heading_deg']:5.1f}° {r['stroke']}: {r['meaning']} | "
162             f"{omega_words(r)} | G.n≤0 {'yes' if r['inward'] else 'NO'} | "
163             f"L {r['ledger']:.4f}{step} | axle S {r['axle_S']} (rest {r['axle_rest']}) | circles {r['circles_closed']}")
    )
```

```

==== omniconpass/configurations.py (68 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Named engine configurations.
5
6  Default remains omniconpass.core (symmetric_verified). This module does not
7  replace it. printed_eight_line is the marketing-plate transcription.
8
9  symmetric_verified claim-2 / package core
10     dE = -alpha_E E + beta_int + beta_ext + v_eff
11     dU = mu U (1-U^2) - dE/E_max - lambda_U U + u
12     dI = 1-U - sigma_1 E - delta S - lambda_I I_U
13     v = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U-0.5) + lambda_2 S)
14
15     printed_eight_line plate species (not the held-out evidence core)
16     dE = -alpha E + beta_int + beta_ext + v_eff
17     dU = alpha (1-U) - dE/E_max - k U (1-U) + u
18     dI = 1-U - sigma_1 E - delta S
19     v = c * tanh(lambda_0 + lambda_1 (U-U_t) + lambda_2 S)
20 """
21 from __future__ import annotations
22
23 import math
24 from typing import Callable
25
26 from .core import EPS, KP, U_AUTHORITY, Params, State, derivatives as core_derivatives, v_eff as core_v_eff
27
28
29 SYMMETRIC = "symmetric_verified"
30 PRINTED = "printed_eight_line"
31 DEFAULT = SYMMETRIC
32
33
34 def printed_v_eff(x: State, p: Params, t: float = 0.0, U_t: float = 0.5) -> float:
35     z = p.lambda_0 + p.lambda_1 * (x.U - U_t) + p.lambda_2 * x.S
36     return p.c * math.tanh(z)
37
38
39 def printed_derivatives(x: State, p: Params, t: float, u: float = 0.0, U_t: float = 0.5) -> State:
40     v = printed_v_eff(x, p, t, U_t)
41     dE = -p.alpha * x.E + p.beta_int + p.beta_ext + v
42     dU = p.alpha * (1.0 - x.U) - dE / max(p.E_max, 1e-6) - p.k * x.U * (1.0 - x.U) + u
43     dI = (1.0 - x.U) - p.sigma_1 * x.E - p.delta * x.S
44     dS = p.delta - p.alpha_s * x.S - 0.75 * p.beta_s * x.S * x.S
45     dBd = p.gamma_c * p.delta * x.S - (p.omega_B / max(p.Q_B, EPS)) * x.B_dot - p.omega_B ** 2 * x.B
46     return State(dE, dU, dI, dS, x.B_dot, dBd)
47
48
49 def derivatives_for(name: str) -> Callable:
50     if name == SYMMETRIC:
51         return core_derivatives
52     if name == PRINTED:
53         return printed_derivatives
54     raise ValueError(name)
55
56
57 def control_command_for(name: str, x: State, p: Params, t: float, target: float):
58     deriv = derivatives_for(name)
59     drift = deriv(x, p, t, 0.0).U
60     raw = -drift + KP * (float(target) - x.U)
61     return max(-U_AUTHORITY, min(U_AUTHORITY, raw)), raw
62
63
64 def describe(name: str) -> str:
65     return {
66         SYMMETRIC: "package / claim-2 cubic U; owns held-out evidence",
67         PRINTED: "plate logistic U; executable; not the default",
68     }[name]

```

```

==== omniconpass/conveyance.py (101 lines) ====
1  """Conveyance law: moving a conserved budget ("juice") between organs.
2
3  Manuscript sources (interpretation of the English, not only the symbols):
4  Ch. 29 §8   E_system = E + U + S + B, dE_system/dt = 0 over the cycle: "Energy redistributes. Energy reorganizes.
5              Energy does not diverge." -> the budget is conserved, only moved
6  Ch. 31 §4   "When E grows locally, B increases to distribute gradients." -> flow runs down the gradient of need
7  Ch. 31 §5   stability requires Re(lambda_i) <= 0 -> the flow must provably converge
8  Ch. 30 §5-7 ignition is a "redistribution surge ... controlled release", with state continuity (no discontinuity)
9              -> the reserve is released continuously
10 Ch. 29 §6   compression: "backpressure develops through S ... structured inward rotation"
11              -> surplus is pulled back smoothly
12
13 The law. Organs i hold allocations a_i of a budget A (watts, cores, ...), have need d_i (the budget that would serve
14 their work at the engine's utilisation target rho), and a deficit ("local deviation")
15   e_i = d_i / (rho a_i) - 1      (> 0 needs juice, < 0 holds surplus)
16 The redistribution mediator moves budget by the replicator flow
17   da_i/dt = kappa a_i (e_i - e_bar),   e_bar = sum_j a_j e_j / A
18 Properties (proved in docs/CONVEYANCE_LAW.md, checked in tests/test_conveyance.py):
19 P1 conservation   sum_i da_i/dt = 0 exactly: the budget is moved, never created
20 P2 gradient flow  it is the Shahshahani gradient of F(a) = sum_i (d_i / rho) ln a_i on the simplex sum a_i = A;
21                   F is strictly concave, so F rises monotonically (Lyapunov function)
22 P3 convergence    unique equilibrium a_i* = A d_i / D (D = sum d_i): every organ at the same deficit e*,
23                   reached exponentially: da_i/dt = (kappa/rho)(d_i - a_i D/A), rate kappa D / (rho A), and the
24                   linearisation has eigenvalues -kappa D/(rho A) < 0 (Re(lambda) <= 0, Ch. 31 §5)
25 P4 bounds         organ floors and ceilings (device minimum, TDP) are kept by projection; budget an organ cannot
26                   take because it is at its ceiling returns to the others (water-filling), and budget left over
27                   when every organ is at its need is held back as reserve, not spent
28 Reserve (Ch. 30 surge): R = the budget needed to cover the demand rise the rate tracker projects over one actuation
29 delay; released continuously to organs whose deficit stays positive after the flow."""
30 from __future__ import annotations
31
32 from typing import List, Sequence
33
34
35 def deficits(a: Sequence[float], d: Sequence[float], rho: float) -> List[float]:
36     return [di / (rho * max(ai, 1e-12)) - 1.0 for ai, di in zip(a, d)]
37
38
39 def replicator_step(a: Sequence[float], d: Sequence[float], rho: float, kappa: float, dt: float = 1.0) -> List[float]:
40     """One exact step of da_i/dt = (kappa/rho)(d_i - a_i D/A), the closed form of the replicator flow (P3):
41     a_i(t+dt) = a_i* + (a_i - a_i*) exp(-kappa D dt / (rho A)); conserves sum a_i to rounding (P1)."""
42     import math
43     A = sum(a); D = sum(d)
44     if A <= 0 or D <= 0:
45         return list(a)
46     k = math.exp(-kappa * D * dt / (rho * A))
47     return [A * di / D + (ai - A * di / D) * k for ai, di in zip(a, d)]
48
49
50 def project(a: Sequence[float], lo: Sequence[float], hi: Sequence[float], budget: float) -> List[float]:
51     """Water-filling projection onto {lo_i <= a_i <= hi_i, sum a_i <= budget}: organs pinned at a bound keep it, the
52     rest share the remainder in proportion to their unconstrained allocation (P4). Exact in <= n rounds."""
53     a = [min(max(x, l), h) for x, l, h in zip(a, lo, hi)]
54     for _ in range(len(a) + 1):
55         tot = sum(a)
56         if tot <= budget + 1e-9:
57             return a
58         free = [i for i in range(len(a)) if a[i] > lo[i] + 1e-12]
59         over = tot - budget
60         share = sum(a[i] - lo[i] for i in free)
61         if share <= 0:
62             return a
63         a = [max(lo[i], a[i] - over * (a[i] - lo[i]) / share) if i in free else a[i] for i in range(len(a))]
64     return a
65
66
67 class Conveyance:
68     """The budget law for a set of organs sharing one budget (e.g. GPU groups under one site power cap).
69     step(need, lo, hi) returns allocations; the flow moves budget toward need, the projection keeps bounds, and the
70     reserve holds back what the projected rise needs, releasing it continuously where deficits stay positive."""
71
72     def __init__(self, n: int, budget: float, rho: float = 0.8, kappa: float = 1.0):
73         self.budget, self.rho, self.kappa = budget, rho, kappa
74         self.a = [budget / n] * n
75         self.prev = None
76         self.rate = [0.0] * n
77
78     def step(self, need: Sequence[float], lo: Sequence[float], hi: Sequence[float]) -> List[float]:
79         # the living band: every organ keeps at least 5% of its range (idle, never off) and never takes more than 95%
80         from omniconpass.nervous_system import BAND

```

```

81     lo = [max(l, BAND[0] * h) for l, h in zip(lo, hi)]
82     hi = [max(l, BAND[1] * h) for l, h in zip(lo, hi)]
83     need = [max(x, 1e-9) for x in need]
84     if self.prev is not None: # rate tracker for the reserve (the dual-bath rate channel)
85         self.rate = [0.5 * r + 0.5 * (x - p) for r, x, p in zip(self.rate, need, self.prev)]
86     self.prev = list(need)
87     reserve = min(0.5 * self.budget, sum(max(0.0, r) for r in self.rate))
88     want = [min(h, x / self.rho) for x, h in zip(need, hi)] # budget each organ can use at rho
89     want = [max(w, l) for w, l in zip(want, lo)] # a device cannot go below its floor
90     spend = max(sum(lo), min(self.budget - reserve, sum(want))) # never spend budget nobody needs
91     a = replicator_step(self.a, [w * self.rho for w in want], self.rho, self.kappa)
92     a = [x * spend / max(sum(a), 1e-12) for x in a]
93     a = project(a, lo, [max(l, min(h, w)) if sum(want) <= spend else h for l, h, w in zip(lo, hi, want)], spend)
94     # surge (Ch. 30): release the reserve continuously to organs still in deficit, up to their ceiling
95     left = self.budget - sum(a)
96     short = [max(0.0, min(h, x / self.rho) - ai) for x, h, ai in zip(need, hi, a)]
97     tot = sum(short)
98     if tot > 0 and left > 0:
99         a = [ai + min(s, left * s / tot) for ai, s in zip(a, short)]
100     self.a = a
101     return a

```

```

==== omnicompass/core.py (240 lines) ====
1  """Omni-Compass core: six-state ODE, RK4 integrator, bounded feedforward-plus-proportional controller.
2
3  State x = (E, U, I_U, S, B, B_dot).
4
5  (1) dE/dt = -alpha_E E + beta_int + beta_ext + v_eff
6  (2) dU/dt = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u, |u| <= U_AUTHORITY
7  (3) dI_U/dt = (1 - U) - sigma_1 E - delta S - lambda_I I_U
8  (4) v_eff = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
9  (5) Phi(S) = alpha_s S^2 / 2 + beta_s S^3 / 4 - delta S
10 (6) dS/dt = -dPhi/dS = delta - alpha_s S - (3/4) beta_s S^2
11 (7) dB/dt = B_dot ; dB_dot/dt = gamma_c delta S - (omega_B/Q_B) B_dot - omega_B^2 B
12 (8) R_B[n] = finite-difference audit of (7); never fed back into the state.
13
14 Controller (one micro step, zero-order hold across all four RK4 stages):
15 u = clip(-f_U(x, t) + KP (sigma - U), -U_AUTHORITY, +U_AUTHORITY)
16 where f_U is the uncontrolled U drift and sigma in {-1, +1} is the locked target.
17 """
18 from __future__ import annotations
19
20 import math
21 from dataclasses import dataclass, asdict, fields
22 from typing import Dict, List, Optional, Tuple
23
24 EPS = 1e-12
25 KP = 12.0
26 U_AUTHORITY = 25.0
27 MACRO_STEPS = 20
28 MICRO_STEPS = 10
29 MACRO_DT = 0.1
30 MICRO_DT = MACRO_DT / MICRO_STEPS
31 BASIN_TOL = 0.10
32 CONVEY_WINDOW = 5
33 CERT_WINDOW = 10
34
35 PARAMETER_RANGES: Dict[str, Tuple[float, float]] = {
36     "alpha": (4.2, 4.2), "beta_int": (0.0, 1.0), "beta_ext": (0.0, 1.0), "k": (1.7, 1.7),
37     "sigma_1": (0.38, 0.38), "delta": (0.01, 1.0), "gamma_1": (1.35, 1.35), "gamma_c": (1.0, 1.0),
38     "lambda_0": (-2.0, 2.0), "lambda_1": (0.1, 3.0), "lambda_2": (0.0, 2.0), "c": (0.5, 5.0),
39     "E_max": (1.0, 10.0), "omega_B": (0.1, 5.0), "Q_B": (0.5, 10.0), "alpha_s": (0.05, 0.25),
40     "beta_s": (0.05, 0.25), "mu": (2.0, 6.0), "alpha_E": (4.2, 4.2), "alpha_U": (4.2, 4.2),
41     "lambda_I": (0.5, 2.0), "lambda_U": (0.0, 0.5),
42 }
43 # alpha, k, gamma_1, alpha_U do not enter equations (1)-(8).
44
45
46 @dataclass
47 class State:
48     E: float
49     U: float
50     I_U: float
51     S: float
52     B: float
53     B_dot: float
54
55     def as_tuple(self) -> Tuple[float, ...]:
56         return (self.E, self.U, self.I_U, self.S, self.B, self.B_dot)
57
58
59 @dataclass
60 class Params:
61     alpha: float = 4.2
62     beta_int: float = 0.0
63     beta_ext: float = 0.0
64     k: float = 1.7
65     sigma_1: float = 0.38
66     delta: float = 0.5
67     gamma_1: float = 1.35
68     gamma_c: float = 1.0
69     lambda_0: float = 0.0
70     lambda_1: float = 1.0
71     lambda_2: float = 0.5
72     c: float = 1.0
73     E_max: float = 2.0
74     omega_B: float = 1.0
75     Q_B: float = 2.5
76     alpha_s: float = 0.12
77     beta_s: float = 0.10
78     mu: float = 4.0
79     alpha_E: float = 4.2
80     alpha_U: float = 4.2

```

```

81     lambda_I: float = 1.0
82     lambda_U: float = 0.1
83
84
85 # ----- equations
86 def v_eff(x: State, p: Params, t: float = 0.0) -> float:
87     z = p.lambda_0 + p.lambda_1 * (x.U - 0.5) + p.lambda_2 * x.S
88     return math.cos(0.5 * p.omega_B * t) * (p.c * math.tanh(z))
89
90
91 def phi(S: float, p: Params) -> float:
92     return 0.5 * p.alpha_s * S * S + 0.25 * p.beta_s * S ** 3 - p.delta * S
93
94
95 def s_flow(S: float, p: Params) -> float:
96     return p.delta - p.alpha_s * S - 0.75 * p.beta_s * S * S
97
98
99 def derivatives(x: State, p: Params, t: float, u: float = 0.0) -> State:
100     v = v_eff(x, p, t)
101     dE = -p.alpha_E * x.E + p.beta_int + p.beta_ext + v
102     dU = p.mu * x.U * (1.0 - x.U * x.U) - dE / max(p.E_max, 1e-6) - p.lambda_U * x.U + u
103     dI = (1.0 - x.U) - p.sigma_1 * x.E - p.delta * x.S - p.lambda_I * x.I_U
104     dS = s_flow(x.S, p)
105     dBd = p.gamma_c * p.delta * x.S - (p.omega_B / max(p.Q_B, EPS)) * x.B_dot - p.omega_B ** 2 * x.B
106     return State(dE, dU, dI, dS, x.B_dot, dBd)
107
108
109 def _axpy(x: State, k: State, h: float) -> State:
110     return State(x.E + h * k.E, x.U + h * k.U, x.I_U + h * k.I_U, x.S + h * k.S, x.B + h * k.B, x.B_dot + h * k.B_dot)
111
112
113 def rk4_step(x: State, p: Params, t: float, h: float, u: float = 0.0) -> State:
114     k1 = derivatives(x, p, t, u)
115     k2 = derivatives(_axpy(x, k1, 0.5 * h), p, t + 0.5 * h, u)
116     k3 = derivatives(_axpy(x, k2, 0.5 * h), p, t + 0.5 * h, u)
117     k4 = derivatives(_axpy(x, k3, h), p, t + h, u)
118     c = h / 6.0
119     return State(*(a + c * (b1 + 2 * b2 + 2 * b3 + b4) for a, b1, b2, b3, b4 in
120                     zip(x.as_tuple(), k1.as_tuple(), k2.as_tuple(), k3.as_tuple(), k4.as_tuple())))
121
122
123 def bath_residual(prev: State, cur: State, nxt: State, p: Params, h: float) -> float:
124     d2 = (nxt.B - 2.0 * cur.B + prev.B) / (h * h)
125     dm = (cur.B - prev.B) / h
126     return d2 + (p.omega_B / max(p.Q_B, EPS)) * dm + p.omega_B ** 2 * cur.B - p.gamma_c * p.delta * cur.S
127
128
129 # ----- controller
130 def control_command(x: State, p: Params, t: float, target: int) -> Tuple[float, float]:
131     """Return (u, raw) for one micro step. raw is the unclipped command."""
132     drift = derivatives(x, p, t, 0.0).U
133     raw = -drift + KP * (float(target) - x.U)
134     return max(-U_AUTHORITY, min(U_AUTHORITY, raw)), raw
135
136
137 def basin_sign(U: float) -> int:
138     if abs(U - 1.0) <= BASIN_TOL:
139         return 1
140     if abs(U + 1.0) <= BASIN_TOL:
141         return -1
142     return 0
143
144
145 def nearest_basin_sign(U: float) -> int:
146     return 1 if abs(U - 1.0) <= abs(U + 1.0) else -1
147
148
149 def macro_step(x: State, p: Params, t0: float, target: Optional[int],
150               micro_steps: int = MICRO_STEPS, macro_dt: float = MACRO_DT) -> Tuple[State, Dict[str, float]]:
151     """Advance one macro interval. target=None runs the plant with u=0 (observe)."""
152     h = macro_dt / micro_steps
153     t = t0
154     u_abs = u_sq = u_peak = 0.0
155     sat = 0
156     for _ in range(micro_steps):
157         if target is None:
158             u = raw = 0.0
159         else:
160             u, raw = control_command(x, p, t, target)
161         u_abs += abs(u) * h

```

```

162         u_sq += u * u * h
163         u_peak = max(u_peak, abs(u))
164         sat += int(abs(raw) > U_AUTHORITY + EPS)
165         x = rk4_step(x, p, t, h, u)
166         t += h
167     return x, {"u_abs": u_abs, "u_sq": u_sq, "u_peak": u_peak, "saturated": sat}
168
169
170 # ----- trajectory
171 @dataclass
172 class Result:
173     final: State
174     U_hist: List[float]
175     target_sign: int
176     born: int
177     convey: int
178     convey_start: int
179     convey_confirm: int
180     cert: int
181     cert_start: int
182     cert_complete: int
183     max_streak: int
184     actuator_abs_integral: float
185     actuator_sq_integral: float
186     actuator_peak: float
187     actuator_saturated_periods: int
188
189
190 def simulate(x0: State, p: Params, target: Optional[int] = None, controlled: bool = True) -> Result:
191     """Canonical 20-macro-step trajectory with CONVEY-5 / CERT-10 predicates.
192
193     target defaults to the nearest basin of U0. controlled=False runs u=0.
194     """
195     tgt = nearest_basin_sign(x0.U) if target is None else int(target)
196     b0 = basin_sign(x0.U)
197     born = int(b0 != 0)
198     streak = 1 if born else 0
199     ssign = b0 if born else 0
200     best = streak
201     convey = cert = 0
202     cs = cc = ts = tc = -1
203     ua = usq = upk = 0.0
204     nsat = 0
205     x = x0
206     hist = [x0.U]
207     for m in range(1, MACRO_STEPS + 1):
208         x, a = macro_step(x, p, (m - 1) * MACRO_DT, tgt if controlled else None)
209         ua += a["u_abs"]; usq += a["u_sq"]; upk = max(upk, a["u_peak"]); nsat += a["saturated"]
210         hist.append(x.U)
211         b = basin_sign(x.U)
212         if b:
213             if b == ssign:
214                 streak += 1
215             else:
216                 ssign, streak = b, 1
217         else:
218             ssign, streak = 0, 0
219         best = max(best, streak)
220         if not convey and streak >= CONVEY_WINDOW:
221             convey, cs, cc = 1, m - CONVEY_WINDOW + 1, m
222         if not cert and streak >= CERT_WINDOW:
223             cert, ts, tc = 1, m - CERT_WINDOW + 1, m
224     return Result(x, hist, tgt, born, convey, cs, cc, cert, ts, tc, best, ua, usq, upk, nsat)
225
226
227 def sample_state(rng) -> State:
228     return State(float(rng.uniform(0.0, 1.0)), float(rng.uniform(-1.0, 1.0)), float(rng.uniform(0.0, 1.0)),
229                 float(rng.uniform(-1.0, 1.0)), float(rng.uniform(-1.0, 1.0)), float(rng.uniform(-1.0, 1.0)))
230
231
232 def sample_params(rng) -> Params:
233     return Params(**{k: float(rng.uniform(lo, hi)) for k, (lo, hi) in PARAMETER_RANGES.items()})
234
235
236 def s_roots(p: Params) -> Tuple[float, float]:
237     """Roots of equation (6): S_minus < 0 < S_plus."""
238     a = 0.75 * p.beta_s
239     d = math.sqrt(p.alpha_s ** 2 + 4.0 * a * p.delta)
240     return (-p.alpha_s - d) / (2.0 * a), (-p.alpha_s + d) / (2.0 * a)

```

```
==== omnicompass/disruption_budget.py (29 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Karpenter-shaped voluntary node-down budget.
5
6  Not Karpenter. Caps how many nodes Omni may remove in a window.
7  allowed = ceil(n0 * pct/100), then optional absolute ceiling.
8  """
9  from __future__ import annotations
10 import math
11
12
13 def allowed_downs(n0: int, pct: float, abs_cap: int = 0) -> int:
14     n0 = max(0, int(n0))
15     raw = int(math.ceil(n0 * max(0.0, float(pct)) / 100.0)) if pct else n0
16     if abs_cap and abs_cap > 0:
17         raw = min(raw, int(abs_cap))
18     return max(0, raw)
19
20
21 def apply_node_delta(delta: int, used: int, allow: int) -> tuple[int, int, bool]:
22     """Return (delta, used, blocked). Only clips negative (voluntary shrink)."""
23     if delta >= 0:
24         return int(delta), int(used), False
25     room = max(0, int(allow) - int(used))
26     if room <= 0:
27         return 0, int(used), True
28     take = max(delta, -room) # delta is negative
29     return int(take), int(used) - int(take), False
```

```

==== omnicompass/master.py (144 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """The master switch: one OFF for the whole harness, not one muscle at a time.
5
6  Every Omni-Compass governor on a machine (the Kubernetes controller, the GPU governors, any muscle they drive) obeys on
7  switch, in a human hand:
8
9  OFF  the switch file exists (OMNI_MASTER_OFF, by default /tmp/omni-compass/OFF). Every running governor sees it at i
10 ts
11     next decision, puts every setting it ever wrote back to the value it read before its first write, reads each on
12 e
13     back, writes its last audit line and exits. No governor starts while the switch is OFF.
14 ON   the switch file is removed. Nothing restarts by itself: a governor runs again only when someone starts it.
15
16 The switch is also pulled from the command line, which signals every registered governor at once so none waits for its
17 next decision:
18
19     python3 tools/omni_switch.py off [--reason TEXT]    turn the whole harness off, wait for every governor to restore
20                                                         allow governors to be started again
21     python3 tools/omni_switch.py on                    OFF or ON, and every governor running
22     python3 tools/omni_switch.py status
23
24 A governor that cannot read the switch (a filesystem it cannot reach) treats that as OFF: when in doubt, hand back.
25
26 If a governor dies without handing back (killed outright, the machine crashed, it hung), nothing it wrote may be left
27 in place. So every governor records, before its first write, the exact commands that put every setting back, and
28 renews a lease every decision. The watchdog finds any governor whose process is gone or whose lease is older than its
29 limit and runs those commands for it:
30
31     python3 tools/omni_switch.py watchdog [--stale 60] [--every 10]    keep watching (run it as its own service)
32     python3 tools/omni_switch.py watchdog --once                      one pass
33 """
34 from __future__ import annotations
35
36 import atexit
37 import json
38 import os
39 import time
40
41 DEFAULT_DIR = "/tmp/omni-compass"
42
43 def switch_path() -> str:
44     return os.environ.get("OMNI_MASTER_OFF", os.path.join(DEFAULT_DIR, "OFF"))
45
46 def registry_dir() -> str:
47     return os.environ.get("OMNI_REGISTRY", os.path.join(os.path.dirname(switch_path()) or DEFAULT_DIR, "running"))
48
49 def is_off() -> bool:
50     """True when the master switch is OFF (or cannot be read: when in doubt, hand back)."""
51     p = switch_path()
52     try:
53         return os.path.exists(p)
54     except OSError:
55         return True
56
57 def reason() -> str:
58     try:
59         return json.loads(open(switch_path()).read()).get("reason", "")
60     except (OSError, ValueError):
61         return ""
62
63 _ME = {}
64
65 def register(name: str, restore=None, stale_s: float = 60.0) -> str:
66     """Record this governor as running, so the switch can signal it, with the commands (argument lists) that put every
67     setting back if it dies without doing so itself; removed when the process exits cleanly."""
68     d = registry_dir()
69     os.makedirs(d, exist_ok=True)
70     path = os.path.join(d, f"{os.getpid()}.json")
71     rec = {"pid": os.getpid(), "name": name, "started": time.time(), "lease": time.time(), "stale_s": stale_s,
72           "restore": restore or []}
73     with open(path + ".tmp", "w") as f:
74         json.dump(rec, f)

```

```

74     contradictions = {v: {c: float(np.mean([both[(v, s)][1][c] for s in SEEDS])) for c in COMPETITORS + ["omni"]}} for v
    in VESSELS}
75     rng = np.random.default_rng(20260927)
76     cells = []
77     for v in VESSELS:
78         for c in COMPETITORS:
79             for m in LOWER + HIGHER:
80                 a = np.array([rows[(v, s)][c][m] for s in SEEDS]); b = np.array([rows[(v, s)]["omni"][m] for s in SEEDS
    ])
81                 d = (a - b) if m in LOWER else (b - a) # positive = Omni better
82                 bs = d[rng.integers(0, len(d), (10000, len(d)))].mean(1)
83                 p = float(min(1.0, 2 * min((bs <= 0).mean(), (bs >= 0).mean()))))
84                 scale = max(abs(a.mean()), 1e-9)
85                 cells.append({"vessel": v, "competitor": c, "gauge": m, "platform": float(a.mean()), "omni": float(b.me
    an()),
86                             "omni_better_by_pct": float(d.mean() / scale * 100), "p": p,
87                             "practical": bool(abs(d.mean()) > (0.5 if m in COUNTS else TOL * scale))})
88     order = sorted(range(len(cells)), key=lambda i: cells[i]["p"]); m = len(cells); stop = False
89     for k, i in enumerate(order): # Holm-Bonferroni step-down
90         thr = 0.05 / (m - k)
91         cells[i]["holm_significant"] = (not stop) and cells[i]["p"] <= thr
92         stop = stop or not cells[i]["holm_significant"]
93     for c_ in cells:
94         c_["verdict"] = ("better" if c_["omni_better_by_pct"] > 0 else "worse") if (c_["holm_significant"] and c_["prac
    tical"]) else "equal"
95     from collections import Counter
96     tally = Counter(c_["verdict"] for c_ in cells)
97     per = {v: dict(Counter(c_["verdict"] for c_ in cells if c_["vessel"] == v)) for v in VESSELS}
98     out = {"setting": SET, "preregistration_sha256": PRE["sha256"], "seeds": [SEEDS[0], SEEDS[-1]], "tally": dict(tally
    ), "per_workload": per,
99           "held_out_rounds": ROUNDS, "cells": cells}
100     if COORD:
101         out["coord"] = True; out["contradictions_mean_per_day"] = contradictions
102         (ROOT / ("tuning/CONFIRMATORY_COORD.json" if COORD else "tuning/CONFIRMATORY.json")).write_text(json.dumps(out, ind
    ent=1))
103     print("confirmatory" + (" with coordination" if COORD else "") + ", 100 scenarios per workload, Holm-corrected:", d
    ict(tally))
104     if COORD:
105         for v in VESSELS:
106             print(" contradictions", v, {k: round(x, 2) for k, x in contradictions[v].items()})
107         for v in VESSELS:
108             print(" ", v, per[v], [(c_["competitor"], c_["gauge"], round(c_["omni_better_by_pct"], 1)) for c_ in cells if c
    _["vessel"] == v and c_["verdict"] == "worse"])

```

```

==== tuning/dominate.py (103 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Can Omni-Compass be equal or better than EACH competitor on EVERY gauge? (development seeds 101-110)
5
6  Two questions, answered from one search:
7      individually for each competitor and vessel: is there an Omni configuration with no losing gauge against it?
8      all at once for each vessel: is there one Omni configuration with no losing gauge against every competitor?
9  Candidate Omni configurations: the frozen fleet-law family with free constants, and the speed-law family
10 (omnicompass/speed.py); the engine equations are unchanged. A loss is as in tuning/league.py (worse beyond 0.5% and the
11 paired bootstrap 95% interval excludes zero).
12 Usage: python tuning/dominate.py N SEED
13 """
14 from __future__ import annotations
15
16 import json, random, sys
17 from dataclasses import replace
18 from multiprocessing import Pool
19 from pathlib import Path
20
21 import numpy as np
22
23 ROOT = Path(__file__).resolve().parents[1]
24 sys.path.insert(0, str(ROOT))
25 from fleet import sim_slo
26 from fleet.harness import make_scenario
27 from omnicompass.adapter import mode_law
28 from omnicompass.speed import SpeedLaw
29 from tuning.speed_search import gauges, LOWER, HIGHER
30 from tuning.league import COMPETITORS, VESSELS, SEEDS, losses
31
32 KEYS = [(v, s) for v in VESSELS for s in SEEDS["dev"]]
33
34
35 def sample(rng, i):
36     if i % 2 == 0:
37         rho0 = rng.uniform(0.50, 0.80)
38         return {"family": "speed", "law": dict(rho0=rho0, rho_min=rng.uniform(0.40, rho0), kI=rng.uniform(0, 0.3),
39         kE=rng.uniform(0, 0.5), kq=rng.uniform(0.0, 1.0), headroom=rng.uniform(0.0, 0.3),
40         headroom_up=rng.uniform(0.0, 0.2), window=rng.randint(1, 60), band=rng.randint(1, 4),
41         dwell=rng.randint(1, 40), after_add=rng.randint(0, 40), push_release=rng.uniform(0.0, 0.15),
42         cap_idle=rng.uniform(0.7, 1.0), cap_busy=rng.uniform(0.05, 0.5)),
43         "every": rng.choice([1, 2, 4]), "lat_gain": rng.uniform(0, 3), "slo_mult": rng.uniform(1.1, 3.0)}
44     rho0 = rng.uniform(0.55, 0.95)
45     return {"family": "fleet", "law": dict(rho0=rho0, rho_min=rng.uniform(0.40, rho0), kI=rng.uniform(0, 0.3),
46     kE=rng.uniform(0, 0.5), kq=rng.uniform(0.2, 1.5), down_band=rng.randint(0, 4), down_dwell=rng.randint(1, 20
47 ),
48     down_after_add=rng.randint(0, 20), push_release=rng.uniform(0.01, 0.12), cap_min=rng.uniform(0.65, 1.0),
49     margin=rng.uniform(0, 0.3), U_gate=rng.uniform(0.4, 0.95), up_max=rng.choice([2, 4, 8])),
50     "every": rng.choice([1, 2, 4]), "lat_gain": rng.uniform(0, 3), "slo_mult": rng.uniform(1.1, 3.0)}
51
52 def run_cfg(sc, cfg):
53     if cfg["family"] == "speed":
54         r = sim_slo.run(sc, "omni_speed", speed_law=SpeedLaw(**cfg["law"]), omni_every=cfg["every"],
55         lat_gain=cfg["lat_gain"], slo_mult=cfg["slo_mult"])
56     else:
57         r = sim_slo.run(sc, "omni_fleet", governor_law=replace(mode_law("throughput"), **cfg["law"]),
58         omni_every=cfg["every"], lat_gain=cfg["lat_gain"], slo_mult=cfg["slo_mult"])
59     return gauges(r)
60
61
62 def _cfg_job(args):
63     i, cfg = args
64     return i, {k: run_cfg(make_scenario(*k), cfg) for k in KEYS}
65
66
67 def _comp_job(k):
68     sc = make_scenario(*k)
69     return k, {c: gauges(sim_slo.run(sc, c)) for c in COMPETITORS}
70
71
72 def slim(g):
73     return {m: float(g[m]) for m in LOWER + HIGHER}
74
75
76 if __name__ == "__main__":
77     n, seed = int(sys.argv[1]), int(sys.argv[2])
78     rng = random.Random(seed)
79     cfgs = {i: sample(rng, i) for i in range(n)}

```

```

80     with Pool(4) as p:
81         comp = dict(p.map(_comp_job, KEYS))
82         res = dict(p.map(_cfg_job, list(cfgs.items()), chunksize=2))
83         brng = np.random.default_rng(11)
84         out = {"cfgs": cfgs, "individual": {}, "all_at_once": {}}
85         for v in VESSELS:
86             seeds = SEEDS["dev"]
87             per = {}
88             for i in cfgs:
89                 rows = {(v, s): dict(comp[(v, s)], omni=res[i][(v, s)]) for s in seeds}
90                 per[i] = {c: losses(rows, seeds, "omni", c, v, brng) for c in COMPETITORS}
91                 for c in COMPETITORS:
92                     best = min(per, key=lambda i: (len(per[i][c]), sum(l["omni_worse_by_pct"] for l in per[i][c])))
93                     out["individual"][f"{v}|{c}"] = {"config": best, "losses": per[best][c]}
94                 tot = {i: sum(len(per[i][c]) for c in COMPETITORS) for i in per}
95                 best = min(tot, key=tot.get)
96                 out["all_at_once"][v] = {"config": best, "n_losses": tot[best], "losses": [l for c in COMPETITORS for l in per[
best][c]]}
97             (ROOT / f"tuning/DOMINATE_{seed}.json").write_text(json.dumps(out, indent=1, default=str))
98             print("INDIVIDUALLY (best Omni configuration against each competitor alone):")
99             for k, r in out["individual"].items():
100                 print(f"    {k:24} config {r['config']:>4}: " + ("(", ".join(f'{l['gauge']}' + {l['omni_worse_by_pct']:.1f}% for l i
n r["losses"]) or "NO LOSING GAUGE"))
101             print("ALL AT ONCE (one Omni configuration per vessel against every competitor):")
102             for v, r in out["all_at_once"].items():
103                 print(f"    {v:6} config {r['config']}: {r['n_losses']} losing cells")

```

```

==== tuning/global_league.py (66 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """One global closure-law setting for every workload (no per-workload table). Candidates: each workload's frozen
5  setting from tuning/CLOSURE_FINAL_DEV.json applied to all four workloads (pods at 0.8 for services; job workloads have
6  no
7  pods to size). Four-cluster sites run the same law on the site total with traffic shift. Selection on 30 development
8  seeds (101-130) by total losing cells against the seven platforms; the pick is frozen by SHA-256 and run once on fresh
9  held-out seeds 703001-703030. Usage: python tuning/global_league.py -> tuning/GLOBAL_LEAGUE.json"""
10 import hashlib, json, sys
11 from multiprocessing import Pool
12 from pathlib import Path
13 import numpy as np
14 ROOT = Path(__file__).resolve().parents[1]; sys.path.insert(0, str(ROOT))
15 from fleet import sim_slo
16 from fleet.harness import make_scenario
17 from fleet.sim_slo import DirectLaw
18 from omnicompass.speed import SpeedLaw
19 from omnicompass.closure import ClosureLaw
20 from tuning.speed_search import gauges
21 from tuning.league import COMPETITORS, VESSELS, LOWER, HIGHER, losses
22
23 F = {k: v for k, v in json.load(open(ROOT / "tuning/CLOSURE_FINAL_DEV.json")).items() if not k.startswith("_")}
24 CANDS = {src: {"closure": F[src]["closure"], "direct": F[src]["direct"], "speed": F[src]["speed"]} for src in F}
25 DEV, HELD = list(range(101, 131)), list(range(703001, 703031))
26
27 def omni(sc, v, cand):
28     cl = dict(cand["closure"], site=(v == "multi"))
29     return gauges(sim_slo.run(sc, "omni_closure", speed_law=SpeedLaw(**cand["speed"]), omni_every=1,
30                             direct_law=DirectLaw(**cand["direct"]), closure_law=ClosureLaw(**cl)))
31
32 def job(a):
33     v, s, names = a
34     sc = make_scenario(v, s)
35     row = {c: gauges(sim_slo.run(sc, c)) for c in COMPETITORS}
36     for nm in names:
37         row["omni_" + nm] = omni(sc, v, CANDS[nm])
38     return (v, s), row
39
40 def score(rows, seeds, nm):
41     rng = np.random.default_rng(11)
42     return [l for v in VESSELS for c in COMPETITORS for l in losses(rows, seeds, "omni_" + nm, c, v, rng)]
43
44 if __name__ == "__main__":
45     with Pool(4) as p:
46         rows = dict(p.map(job, [(v, s, list(CANDS)) for v in VESSELS for s in DEV], chunksize=1))
47         dev = {nm: score(rows, DEV, nm) for nm in CANDS}
48         for nm, L in dev.items():
49             print("dev global =", nm, "setting:", len(L), "losing cells", flush=True)
50             pick = min(CANDS, key=lambda nm: (len(dev[nm]), sum(l["omni_worse_by_pct"] for l in dev[nm])))
51             pre = {"global_setting_from": pick, "setting": CANDS[pick], "heldout_seeds": HELD}
52             pre["sha256"] = hashlib.sha256(json.dumps(pre, sort_keys=True).encode()).hexdigest()
53             (ROOT / "tuning/GLOBAL_LEAGUE_PREREGISTRATION.json").write_text(json.dumps(pre, indent=1))
54         with Pool(4) as p:
55             rows = dict(p.map(job, [(v, s, [pick]) for v in VESSELS for s in HELD], chunksize=1))
56             L = score(rows, HELD, pick)
57             means = {v: {a: {m: float(np.mean([rows[(v, s)][a][m] for s in HELD])) for m in LOWER + HIGHER}
58                         for a in COMPETITORS + ["omni_" + pick]} for v in VESSELS}
59             print("heldout global", pick, len(L), "losing cells of 364")
60             for v in VESSELS:
61                 print(" ", v, [(l["competitor"], l["gauge"], round(l["omni_worse_by_pct"], 1)) for l in L if l["vessel"] == v])
62             (ROOT / "tuning/GLOBAL_LEAGUE.json").write_text(json.dumps({"preregistration": pre, "dev_losing_cells": {k: len(v)
63             for k, v in dev.items()}},
64                             "heldout_cells": L, "means": means}, indent=1))

```

```

==== tuning/heldout_amendment.py (46 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Run the pre-registered amendment (tuning/PREREGISTRATION_AMENDMENT_2026-09-26.json) once on the new held-out seeds."""
5  import json, sys
6  from pathlib import Path
7  import numpy as np
8  ROOT = Path(__file__).resolve().parents[1]; sys.path.insert(0, str(ROOT))
9  import benchmarks.stack_benchmark as sb
10 from tuning.tune_stack import scenarios, run_arm, LOWER, HIGHER, AllocationLaw
11 P = json.loads((ROOT / "tuning/PREREGISTRATION_AMENDMENT_2026-09-26.json").read_text())
12 V = P["variants"]
13 arms = {"A_k8s": ("k8s", {}), "B_frozen": ("omni_k8s_throughput", {}), "C_frozen": ("omni_direct", {}),
14         "B_wear": ("omni_k8s_throughput", V["B_wear"]["overrides_after_throughput_mode"]),
15         "C_throughput": ("omni_direct_throughput", V["C_throughput"]["overrides"])}
16 out = {"seeds": P["new_held_out_seeds"], "scenarios_per_seed": P["scenarios_per_seed"], "means": {}, "paired": {}}
17 per = {}
18 for seed in P["new_held_out_seeds"]:
19     cfg, scns = scenarios(P["scenarios_per_seed"], seed)
20     for name, (arm, over) in arms.items():
21         rows = [sb.simulate(s, "k8s_ref_70", cfg, AllocationLaw()) for s in scns] if arm == "k8s" else run_arm(arm, over,
22 r, cfg, scns)
23         per.setdefault(name, {})[seed] = rows
24 rng = np.random.default_rng(12345)
25 for name in arms:
26     out["means"][name] = {m: float(np.mean([r[m] for s in P["new_held_out_seeds"] for r in per[name][s]])) for m in LOWER
27 ER + HIGHER}
28 for name in [n for n in arms if n != "A_k8s"]:
29     out["paired"][name] = {}
30     for m in LOWER + HIGHER:
31         cis = []
32         for s in P["new_held_out_seeds"]:
33             d = np.array([c[m] - a[m] for a, c in zip(per["A_k8s"][s], per[name][s])])
34             bs = d[rng.integers(0, len(d), (3000, len(d)))].mean(1); cis.append([float(x) for x in np.percentile(bs, [2
35 .5, 97.5])])
36         up = all(lo > 0 for lo, hi in cis); down = all(hi < 0 for lo, hi in cis)
37         better = (down if m in LOWER else up); worse = (up if m in LOWER else down)
38         out["paired"][name][m] = {"ci95_per_seed": cis, "verdict": "better" if better else "worse" if worse else "not s
39 ignificant"}
40 (ROOT / "tuning/HELDOUT_AMENDMENT.json").write_text(json.dumps(out, indent=1))
41 A = out["means"]["A_k8s"]
42 print(f"{'gauge':22}{'A k8s':>12}" + "".join(f"{n:>24}" for n in arms if n != "A_k8s"))
43 for m in LOWER + HIGHER:
44     line = f"{m:22}{A[m]:12.3f}"
45     for n in [n for n in arms if n != "A_k8s"]:
46         v = out["means"][n][m]; rel = (v - A[m]) / abs(A[m]) * 100 if A[m] else 0.0
47         tag = {"better": "+", "worse": "!", "not significant": " "}[out["paired"][n][m]["verdict"]]
48         line += f"{v:12.3f} {rel:+7.1f}% {tag} "
49     print(line)
50 print("worse:", {n: sum(v["verdict"] == "worse" for v in out["paired"][n].values()) for n in out["paired"]})

```

```

==== tuning/law_v2.py (58 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Prototype law v2: two structural changes to how the engine's directive becomes actions (adapter.directive_to_actions
5  is frozen and is not edited; v2 wraps it).
6
7  1. Replicas (direct actuation) are sized with the engine's own sizing equation, the one it uses for nodes:
8       $r^* = \text{ceil}(r * \text{load} / \rho^* + k_q * q * r)$ 
9      and may rise whenever there is backlog, not only when the node decision points up.
10  2. The power cap moves only when the change exceeds a dead-band AND the equation (2) control push has converged
11      ( $|\text{push}| \leq \text{push\_release}$ ), the same gate the engine applies to releasing capacity; raising the cap is never delayed.
12  """
13  from __future__ import annotations
14
15  import math
16  from omnicompass import adapter as A
17
18  _frozen = A.directive_to_actions
19  CAP_DEADBAND = 0.05
20
21
22  def directive_to_actions_v2(d, stack_state, proposals, cfg, direct_actuation=False, law=None, push=0.0):
23      out = [a for a in _frozen(d, stack_state, proposals, cfg, direct_actuation) if a["action"] not in ("replicas", "power_cap")]
24      or not direct_actuation and a["action"] == "replicas"]
25      if direct_actuation:
26          cur_r = int(stack_state["replicas"])
27          q = float(d.get("q", 0.0))
28          kq = law.kq if law is not None else 0.795
29          rt = max(20, min(500, math.ceil(cur_r * float(d["load"]) / max(float(d["demand"]), 1e-9) + kq * q * cur_r)))
30          if rt > cur_r or (rt < cur_r and d["node_delta"] <= 0 and q < 0.05):
31              out.append({"manager": "Omni-Compass", "action": "replicas", "target": rt, "direction": 1 if rt > cur_r else
32  e -1})
33          cur = float(stack_state["power_cap"]); cap = float(d["power_cap"])
34          if any(a["direction"] > 0 and a["action"] in ("nodes", "terraform_plan", "replicas") for a in out):
35              cap = max(cap, cur)
36          converged = law is None or abs(push) <= law.push_release
37          if cap > cur + 1e-9 or (cur - cap > CAP_DEADBAND and converged):
38              out.append({"manager": "Omni-Compass", "action": "power_cap", "target": cap, "direction": -1 if cap < cur else
39  1})
40      return out
41
42  class GovernorV2(A.Governor):
43      """Frozen Governor; the directive also carries the normalised queue, the control push and the law for v2 actuation.
44      """
45      def step(self, obs, conflicts):
46          d = super().step(obs, conflicts)
47          d["q"] = A.observe_vector(obs, conflicts)["q"]; d["push"] = self.last_push; d["_law"] = self.law
48          return d
49
50  def install(sb):
51      """Switch a stack_benchmark module to v2 actuation; returns an uninstall function."""
52      old = (sb.Governor, sb.directive_to_actions)
53      sb.Governor = GovernorV2
54      sb.directive_to_actions = lambda d, st, props, cfg, direct_actuation=False: directive_to_actions_v2(
55          d, st, props, cfg, direct_actuation, d.get("_law"), d.get("push", 0.0))
56      def uninstall():
57          sb.Governor, sb.directive_to_actions = old
58      return uninstall

```

```

==== tuning/league.py (106 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """League table: every competitor against Omni-Compass, one at a time, on every gauge and every vessel.
5
6  Rule (the owner's): Omni-Compass must be equal or better than EACH competitor on EACH gauge. A cell is a loss when the
7  competitor is better by more than the tolerance AND the paired bootstrap 95% interval excludes zero. Every loss goes on
8  the attack list.
9
10 Competitors, each at its documented defaults (fleet plant with response time, fleet/sim_slo.py; vendor arms emulate
11 documented behaviour, not the vendors' binaries):
12     k8s_hpa70_ca    Kubernetes: HPA (70%, the target of every earlier study) + Cluster Autoscaler (= GKE balanced)
13     openshift, gke_optimize, aks_nap (= Karpenter, also EKS Auto Mode), turbonomic, cast_ai, spot_ocean
14 Omni-Compass: one configuration per vessel type (web, multi, batch, gpu), chosen from the candidates below as the one w
15 ith
16 the fewest losses on the development seeds. The operator declaring the vessel type is a human-set boundary.
17 Usage: python tuning/league.py [dev|heldout] [candidates.json]
18 """
19
20 from __future__ import annotations
21
22 import json, sys
23 from multiprocessing import Pool
24 from pathlib import Path
25
26 import numpy as np
27
28 ROOT = Path(__file__).resolve().parents[1]
29 sys.path.insert(0, str(ROOT))
30 from fleet import sim_slo
31 from fleet.harness import make_scenario
32 from omnicompass.speed import SpeedLaw
33 from tuning.speed_search import gauges, LOWER, HIGHER, COUNTS
34
35 COMPETITORS = ["k8s_hpa70_ca", "openshift", "gke_optimize", "aks_nap", "turbonomic", "cast_ai", "spot_ocean"]
36 SENSITIVITY = ["k8s_hpa50_ca", "k8s_hpa60_ca", "k8s_hpa80_ca"] # other Kubernetes HPA targets: reported, not opponent
37 s
38 VESSELS = ("web", "multi", "batch", "gpu")
39 SEEDS = {"dev": list(range(101, 111)), "heldout": list(range(700101, 700131))}
40 TOL = 0.005
41
42 def candidates(path=None):
43     c = {"omni_fleet": None}
44     R = {r["i"]: r["cfg"] for r in json.load(open(ROOT / "tuning/SPEED_SEARCH_21.json"))}
45     for i in (159, 176, 169, 149, 112, 188, 166):
46         c[f"speed{i}"] = R[i]
47     if path:
48         c.update(json.load(open(path)))
49     return c
50
51 def run_arm(sc, name, cfg):
52     if name in COMPETITORS:
53         return gauges(sim_slo.run(sc, name))
54     if cfg is None:
55         return gauges(sim_slo.run(sc, "omni_fleet"))
56     return gauges(sim_slo.run(sc, "omni_speed", speed_law=SpeedLaw(**cfg["law"]), omni_every=cfg["every"],
57         lat_gain=cfg["lat_gain"], slo_mult=cfg["slo_mult"]))
58
59 def _job(args):
60     v, s, cand = args
61     sc = make_scenario(v, s)
62     out = {a: run_arm(sc, a, None) for a in COMPETITORS}
63     out.update({n: run_arm(sc, n, c) for n, c in cand.items()})
64     return (v, s), out
65
66 def losses(rows, seeds, omni, comp, v, rng):
67     L = []
68     for m in LOWER + HIGHER:
69         a = np.array([rows[(v, s)][comp][m] for s in seeds]); b = np.array([rows[(v, s)][omni][m] for s in seeds])
70         d = (b - a) if m in LOWER else (a - b) # positive = Omni worse
71         scale = max(abs(a.mean()), 1e-9)
72         if d.mean() <= TOL * scale + (0.5 if m in COUNTS else 1e-9):
73             continue
74         bs = d[rng.integers(0, len(d), (3000, len(d)))]
75         if np.percentile(bs, 2.5) > 0:
76             L.append({"vessel": v, "competitor": comp, "gauge": m, "competitor_value": float(a.mean()),
77                 "omni_value": float(b.mean()), "omni_worse_by_pct": float(d.mean() / scale * 100)})

```

```

79     return L
80
81
82 def league(which="dev", extra=None, workers=4):
83     cands = candidates(extra)
84     seeds = SEEDS[which]
85     with Pool(workers) as p:
86         rows = dict(p.map(_job, [(v, s, cands) for v in VESSELS for s in seeds], chunksize=1))
87         rng = np.random.default_rng(11)
88         pick, table = {}, {}
89         for v in VESSELS:
90             per = {n: [l for c in COMPETITORS for l in losses(rows, seeds, n, c, v, rng)] for n in cands}
91             best = min(per, key=lambda n: (len(per[n]), sum(l["omni_worse_by_pct"] for l in per[n])))
92             pick[v] = best; table[v] = per[best]
93         means = {v: {a: {m: float(np.mean([rows[(v, s)][a][m] for s in seeds])) for m in LOWER + HIGHER}
94                     for a in COMPETITORS + [pick[v]]} for v in VESSELS}
95     return {"seeds": which, "omni_per_vessel": pick, "attack_list": [l for v in VESSELS for l in table[v]], "means": me
96 ans}
97
98 if __name__ == "__main__":
99     which = sys.argv[1] if len(sys.argv) > 1 else "dev"
100     res = league(which, sys.argv[2] if len(sys.argv) > 2 else None)
101     (ROOT / f"tuning/LEAGUE_{which.upper()}.json").write_text(json.dumps(res, indent=1))
102     print("Omni configuration per vessel:", res["omni_per_vessel"])
103     print(f"attack list: {len(res['attack_list'])} cells where a competitor beats Omni")
104     for l in sorted(res["attack_list"], key=lambda l: (l["vessel"], l["gauge"], -l["omni_worse_by_pct"])):
105         print(f"    {l['vessel']:6} {l['gauge']:18} {l['competitor']:14} competitor {l['competitor_value']:.4g} omni {l[
106 'omni_value']:.4g} "
107             f"(omni worse by {l['omni_worse_by_pct']:.1f}%)")

```

```

==== tuning/math_league.py (44 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """League count for the math-driven governor (omnicompass/mathdrive.py): grid of MathLaw settings per vessel against
5  the seven platforms, development seeds, same loss rule as tuning/league.py. Usage: python tuning/math_league.py"""
6  import itertools, json, sys
7  from multiprocessing import Pool
8  from pathlib import Path
9  ROOT = Path(__file__).resolve().parents[1]; sys.path.insert(0, str(ROOT))
10 from fleet import sim_slo
11 from fleet.harness import make_scenario
12 from omnicompass.mathdrive import MathLaw
13 from tuning.speed_search import gauges
14 from tuning.league import SEEDS, VESSELS
15 from tuning.attack import _comp, score
16
17 GRID = [dict(clock=c, g_w=g, w_release=r, dwell=d, g_cap=gc)
18          for c, g, r, d, gc in itertools.product([0.1, 0.3, 1.0, 3.0], [0.5, 1.0, 2.0], [0.01, 0.05], [3, 10], [0.0, 0.5
19          ])]
20
21 def _eval(args):
22     v, i = args
23     law = MathLaw(**GRID[i])
24     return v, i, {s: gauges(sim_slo.run(make_scenario(v, s), "omni_math", omni_every=4, math_law=law)) for s in SEEDS["
25     dev"]}
26
27 if __name__ == "__main__":
28     with Pool(4) as p:
29         comp = {v: dict(p.map(_comp, [(v, s) for s in SEEDS["dev"]])) for v in VESSELS}
30         res = p.map(_eval, [(v, i) for v in VESSELS for i in range(len(GRID))], chunksize=2)
31         best = {}
32         for v, i, o in res:
33             n, mag, L = score(v, comp[v], o)
34             if v not in best or (n, mag) < best[v][:2]:
35                 best[v] = (n, mag, i, L)
36         out = {v: {"losing_cells": b[0], "setting": GRID[b[2]], "losses": b[3]} for v, b in best.items()}
37         (ROOT / "tuning/MATH_LEAGUE_DEV.json").write_text(json.dumps(out, indent=1))
38         tot = 0
39         for v, b in out.items():
40             tot += b["losing_cells"]
41             print(v, b["losing_cells"], "losing cells, setting", b["setting"])
42             for l in b["losses"]:
43                 print(f"      {l['competitor']:13} {l['gauge']:16} omni worse by {l['omni_worse_by_pct']:.1f}%")
44         print("TOTAL", tot)

```

```

==== tuning/planetlab_league.py (55 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Real demand: recorded PlanetLab VM CPU traces (github.com/beloglazov/planetlab-workload-traces, day 20110303, 1,052
5  machines) as web-service demand (fleet/planetlab.py), Omni-Compass alone with the frozen web closure-law setting
6  (tuning/CLOSURE_FINAL_DEV.json, never tuned on these traces) against the seven platforms, league loss rule, 30 scenario
   s.
7  Also runs the same comparison with whole-pod packing (fleet/sim_slo.py PACKING = "bins").
8  Usage: python tuning/planetlab_league.py /path/to/20110303 -> tuning/PLANETLAB_LEAGUE.json"""
9  import json, sys
10 from multiprocessing import Pool
11 from pathlib import Path
12 import numpy as np
13 ROOT = Path(__file__).resolve().parents[1]; sys.path.insert(0, str(ROOT))
14 from fleet import sim_slo
15 from fleet import planetlab as PL
16 from fleet.sim_slo import DirectLaw
17 from omnicompass.speed import SpeedLaw
18 from omnicompass.closure import ClosureLaw
19 from tuning.speed_search import gauges
20 from tuning.league import COMPETITORS, LOWER, HIGHER, losses
21
22 F = json.load(open(ROOT / "tuning/CLOSURE_FINAL_DEV.json"))["web"]
23 SEEDS = list(range(704001, 704031))
24 TR = None
25
26
27 def job(a):
28     s, packing = a
29     sim_slo.PACKING = packing
30     sc = PL.make_scenario(TR, s)
31     row = {c: gauges(sim_slo.run(sc, c)) for c in COMPETITORS}
32     row["omni"] = gauges(sim_slo.run(sc, "omni_closure", speed_law=SpeedLaw(**F["speed"]), omni_every=1,
33                                     direct_law=DirectLaw(**F["direct"]), closure_law=ClosureLaw(**F["closure"])))
34     return s, row
35
36
37 def init(d):
38     global TR
39     TR = PL.load_dir(d)
40
41
42 if __name__ == "__main__":
43     out = {"trace_dir": "planetlab-workload-traces/20110303", "seeds": SEEDS}
44     for packing in ("fluid", "bins"):
45         with Pool(4, initializer=init, initargs=(sys.argv[1],)) as p:
46             rows = dict(p.map(job, [(s, packing) for s in SEEDS], chunksize=1))
47             rows2 = {("planetlab", s): r for s, r in rows.items()}
48             rng = np.random.default_rng(11)
49             L = [l for c in COMPETITORS for l in losses(rows2, SEEDS, "omni", c, "planetlab", rng)]
50             means = {a: {m: float(np.mean([rows[s][a][m] for s in SEEDS])) for m in LOWER + HIGHER} for a in COMPETITORS +
["omni"]}
51             out[packing] = {"losing_cells": L, "means": means}
52             k = means["k8s_hpa70_ca"]; o = means["omni"]
53             print(packing, "losing cells", len(L), "of 91:", [(l["competitor"], l["gauge"], round(l["omni_worse_by_pct"], 1
54 )) for l in L], flush=True)
55             print("    vs Kubernetes:", {m: f"({k[m] - o[m]} / abs(k[m]) * 100:+.0f)%" for m in ("energy_kwh", "node_hours",
"p95_ms", "p99_ms", "mean_ms", "start_stop")}, flush=True)
56             (ROOT / "tuning/PLANETLAB_LEAGUE.json").write_text(json.dumps(out, indent=1))

```

```

==== tuning/site_league.py (107 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Whole body at multi-cluster sites (four clusters, one site). B: each platform's frozen setting with and without traf
   fic
5  shift; C: the closure law on the site total (one forecast, one warm reserve) with traffic shift, against each incumbent
   .
6  Selection on 30 development seeds (101-130): no losing cell (strict 0.25% for B, league rule for C), then largest gain;
7  frozen by SHA-256, then fresh held-out seeds 701001-701030. Traffic shift assumes each service can run in every cluster
   of
8  the site (replicated services) and adds SHIFT_MS to requests served in another cluster.
9  Usage: python tuning/site_league.py -> tuning/SITE_LEAGUE.json"""
10 import hashlib, itertools, json, sys
11 from multiprocessing import Pool
12 from pathlib import Path
13 import numpy as np
14 ROOT = Path(__file__).resolve().parents[1]; sys.path.insert(0, str(ROOT))
15 import tuning.league as LG
16 from tuning.league import COMPETITORS, losses
17 from tuning.b_league import gains
18 from fleet import sim_slo
19 from fleet.harness import make_scenario
20 from fleet.sim_slo import BLaw, DirectLaw
21 from omnicompass.speed import SpeedLaw
22 from omnicompass.closure import ClosureLaw
23 from tuning.speed_search import gauges
24
25 V = "multi"
26 DEV, HELD = list(range(101, 131)), list(range(701001, 701031))
27 TONE = json.load(open(ROOT / "tuning/B_SETTINGS_TONE.json"))[V]
28 FIX = json.load(open(ROOT / "tuning/B_FIX.json"))["preregistration"]["chosen"]
29 F = json.load(open(ROOT / "tuning/CLOSURE_FINAL_DEV.json"))[V]
30
31
32 def bcands(c):
33     base = dict(FIX.get(f"{V}|{c}", TONE[c]))
34     return [base, dict(base, shift=True)]
35
36
37 CV = [dict(tone_H=th, rho=r, rho_max=rm, turn=tu) for th, r, rm, tu in
38        itertools.product([0, 24, 96, 960], [0.7, 0.72], [0.97, 1.0], [True, False])]
39
40
41 def claw(x):
42     cl = dict(F["closure"], site=True, tone=x["tone_H"] > 0, tone_H=max(1, x["tone_H"]), rho_max=x["rho_max"], turn=x["
   turn"],
43              delta=0.0 if x["rho_max"] >= 1.0 else 0.02, delta_rel=0.0 if x["rho_max"] >= 1.0 else 0.08, H_rel=10, H_a
   dd=2)
44     return cl, dict(F["speed"], rho0=x["rho"], rho_min=x["rho"])
45
46
47 def omni(sc, x):
48     cl, sl = claw(x)
49     return gauges(sim_slo.run(sc, "omni_closure", speed_law=SpeedLaw(**sl), omni_every=1, direct_law=DirectLaw(**F["dir
   ect"])),
50                  closure_law=ClosureLaw(**cl))
51
52
53 def _job(a):
54     s, pickB, pickC = a
55     sc = make_scenario(V, s)
56     A = {c: gauges(sim_slo.run(sc, c)) for c in COMPETITORS}
57     if pickB is None:
58         Bv = {c: [gauges(sim_slo.run(sc, "omniB:" + c, b_law=BLaw(**x)))] for x in bcands(c)] for c in COMPETITORS}
59         Cv = [omni(sc, x) for x in CV]
60     else:
61         Bv = {c: [gauges(sim_slo.run(sc, "omniB:" + c, b_law=BLaw(**pickB[c])))] for c in COMPETITORS}
62         Cv = {c: omni(sc, pickC[c]) for c in COMPETITORS}
63     return s, A, Bv, Cv
64
65
66 def judge(res, seeds, c, getB, strict):
67     LG.TOL = 0.0025 if strict else 0.005
68     rows = {(V, s): {"A": res[s][0][c], "B": getB(s)} for s in seeds}
69     L = losses(rows, seeds, "B", "A", V, np.random.default_rng(11))
70     Aa = {(V, s): {c: res[s][0][c]} for s in seeds}; Bb = {(V, s): {(c, 0): getB(s)} for s in seeds}
71     return L, gains(Aa, Bb, seeds, V, c, 0)
72
73
74 def key(L, g):

```

```

75     return (len(L), -(sum(max(0.0, x) for x in g.values()) - 3 * sum(max(0.0, -x) for x in g.values()))))
76
77
78 if __name__ == "__main__" and "--heldout" in sys.argv:
79     # resume: the settings were already frozen by SHA-256 before any held-out run
80     pre = json.load(open(ROOT / "tuning/SITE_LEAGUE_PREREGISTRATION.json")); pickB, pickC = pre["B"], pre["C"]
81 elif __name__ == "__main__":
82     with Pool(4) as p:
83         r = p.map(_job, [(s, None, None) for s in DEV], chunksize=1)
84         res = {s: (A, Bv, Cv) for s, A, Bv, Cv in r}
85         pickB, pickC = {}, {}
86         for c in COMPETITORS:
87             bb = min(range(2), key=lambda i: key(*judge(res, DEV, c, lambda s: res[s][1][c][i], True)))
88             pickB[c] = bcands(c)[bb]
89             cc = min(range(len(CV)), key=lambda i: key(*judge(res, DEV, c, lambda s: res[s][2][i], False)))
90             pickC[c] = CV[cc]
91             print("dev", c, "B", "shift" if pickB[c].get("shift") else "no shift", "C", CV[cc], flush=True)
92         pre = {"B": pickB, "C": pickC, "heldout_seeds": HELD}
93         pre["sha256"] = hashlib.sha256(json.dumps(pre, sort_keys=True).encode()).hexdigest()
94         (ROOT / "tuning/SITE_LEAGUE_PREREGISTRATION.json").write_text(json.dumps(pre, indent=1))
95 if __name__ == "__main__":
96     with Pool(4) as p:
97         r = p.map(_job, [(s, pickB, pickC) for s in HELD], chunksize=1)
98         res = {s: (A, Bv, Cv) for s, A, Bv, Cv in r}
99         out = {"preregistration": pre, "B": {}, "C": {}}
100        for c in COMPETITORS:
101            L, g = judge(res, HELD, c, lambda s: res[s][1][c][0], False)
102            out["B"][c] = {"losses": L, "gains_pct": g}
103            L2, g2 = judge(res, HELD, c, lambda s: res[s][2][c], False)
104            out["C"][c] = {"losses": L2, "gains_pct": g2}
105            print("heldout", c, "B losses", [(l["gauge"], round(l["omni_worse_by_pct"], 1)) for l in L],
106                  "| C losses", [(l["gauge"], round(l["omni_worse_by_pct"], 1)) for l in L2], flush=True)
107        (ROOT / "tuning/SITE_LEAGUE.json").write_text(json.dumps(out, indent=1))

```

```

==== tuning/speed_search.py (99 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Speed-first fleet mode: random search on development seeds only (101-104 of every vessel).
5
6  Rule, fixed before the search: against HPA 0.7 + Cluster Autoscaler, in EVERY development scenario, NO gauge may be
7  worse (tolerance 0.5% on continuous gauges, 0 on counts). Among configurations that pass, pick the one whose
8  worst gauge-mean improvement (over gauges that can still improve) is largest. Engine equations are unchanged:
9  only law constants and the response-time nerve (lat_gain, slo_mult) are searched.
10 Usage: python tuning/speed_search.py N SEED
11 """
12 import json, random, sys
13 from multiprocessing import Pool
14 from pathlib import Path
15 from dataclasses import replace
16 import numpy as np
17 ROOT = Path(__file__).resolve().parents[1]; sys.path.insert(0, str(ROOT))
18 from fleet.harness import make_scenario
19 from fleet import sim_slo
20 from omnicompass.adapter import mode_law
21 from omnicompass.speed import SpeedLaw
22
23 VESSELS = ("web", "multi", "batch", "gpu")
24 DEV = [(v, s) for v in VESSELS for s in (101, 102, 103, 104)]
25 LOWER = ["energy_kwh", "p95_ms", "p99_ms", "mean_ms", "violation_backlog", "violation_power", "violation_heat",
26          "start_stop", "node_reversals", "node_hours", "pod_changes"]
27 HIGHER = ["work_completed", "time_healthy"]
28 COUNTS = {"start_stop", "node_reversals", "pod_changes"}
29 BASE_ARM = "k8s_hpa70_ca"
30
31
32 def gauges(r):
33     r = dict(r); r["start_stop"] = r["machines_started"] + r["machines_stopped"]; return r
34
35
36 def rel(m, b, o):
37     """Improvement as a fraction of the baseline (positive = better)."""
38     if m in HIGHER:
39         return (o - b) / max(abs(b), 1e-9)
40     return (b - o) / max(abs(b), 1e-9) if b > 1e-12 else (0.0 if o <= 1e-12 else -1.0)
41
42
43 def worse(m, b, o):
44     if m in COUNTS:
45         return o > b
46     if m in HIGHER:
47         return o < b - 0.005 * abs(b) - 1e-9
48     return o > b + 0.005 * abs(b) + 1e-9
49
50
51 def evaluate(cfg, keys=DEV, base=None):
52     rows = {k: gauges(sim_slo.run(make_scenario(*k), "omni_speed", speed_law=SpeedLaw(**cfg["law"]), omni_every=cfg["ev
53     ery"],
54                               lat_gain=cfg["lat_gain"], slo_mult=cfg["slo_mult"])) for k in keys}
55     bad = [(k, m) for k in keys for m in LOWER + HIGHER if worse(m, base[k][m], rows[k][m])]
56     mean_rel = {m: float(np.mean([rel(m, base[k][m], rows[k][m]) for k in keys])) for m in LOWER + HIGHER}
57     return rows, bad, mean_rel
58
59
60 def sample(rng):
61     rho0 = rng.uniform(0.50, 0.75)
62     return {"law": dict(rho0=rho0, rho_min=rng.uniform(0.40, rho0), kI=rng.uniform(0, 0.3), kE=rng.uniform(0, 0.5),
63                        kq=rng.uniform(0.0, 1.0), headroom=rng.uniform(0.0, 0.3), headroom_up=rng.uniform(0.0, 0.2),
64                        window=rng.randint(1, 60), band=rng.randint(1, 4), dwell=rng.randint(1, 40),
65                        after_add=rng.randint(0, 40), push_release=rng.uniform(0.0, 0.15),
66                        cap_idle=rng.uniform(0.7, 1.0), cap_busy=rng.uniform(0.05, 0.5)),
67             "every": rng.choice([1, 2, 4]), "lat_gain": rng.uniform(0, 3), "slo_mult": rng.uniform(1.1, 3.0)}
68
69
70 def _job(args):
71     i, cfg, base = args
72     _, bad, mr = evaluate(cfg, DEV, base)
73     return {"i": i, "cfg": cfg, "n_worse": len(bad), "worse": sorted([m for _, m in bad]), "mean_rel": mr}
74
75
76 def baseline(keys=DEV):
77     return {k: gauges(sim_slo.run(make_scenario(*k), BASE_ARM)) for k in keys}
78
79
80 def score(r, base):

```

```
80     """Worst mean improvement over gauges not already at their floor in every development scenario."""
81     live = [m for m in LOWER + HIGHER if any((base[k][m] > 1e-12 if m in LOWER else base[k][m] < 0.9999) for k in DEV)]
82     return min(r["mean_rel"][m] for m in live)
83
84
85 if __name__ == "__main__":
86     n, seed = int(sys.argv[1]), int(sys.argv[2])
87     base = baseline()
88     rng = random.Random(seed)
89     jobs = [(i, sample(rng), base) for i in range(n)]
90     with Pool(4) as p:
91         res = p.map(_job, jobs, chunksize=4)
92     for r in res:
93         r["score"] = score(r, base)
94     out = ROOT / f"tuning/SPEED_SEARCH_{seed}.json"
95     out.write_text(json.dumps(res, indent=1))
96     ok = [r for r in res if r["n_worse"] == 0]
97     print("configs", len(res), "with no gauge worse anywhere:", len(ok))
98     for r in sorted(res, key=lambda r: (r["n_worse"], -r["score"]))[:8]:
99         print(r["i"], r["n_worse"], r["worse"], round(r["score"], 3), {m: round(v, 3) for m, v in r["mean_rel"].items()})
100 }
```

```

==== tuning/test_c_queue.py (20 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  import sys, itertools
5  from pathlib import Path
6  ROOT = Path(__file__).resolve().parents[1]; sys.path.insert(0, str(ROOT))
7  import benchmarks.stack_benchmark as sb
8  from tuning.tune_stack import scenarios, run_arm, compare, DEV_SEEDS, AllocationLaw
9  sets = [scenarios(40, s) for s in DEV_SEEDS]
10 k8s = [sb.simulate(s, "k8s_ref_70", cfg, AllocationLaw()) for cfg, scns in sets for s in scns]
11 base = {'down_band': 8, 'down_dwell': 10, 'down_after_add': 3, 'U_gate': 0.9, 'cap_min': 0.65}
12 rows_out = []
13 for sfc, rho0, kq, up in itertools.product([True, False], [0.85, 0.75, 0.65], [0.3, 0.795, 1.5], [2, 8]):
14     over = dict(base, size_at_full_cap=sfc, rho0=rho0, rho_min=min(0.627, rho0), kq=kq, up_max=up)
15     c = compare(k8s, [r for cfg, scns in sets for r in run_arm("omni_direct_throughput", over, cfg, scns)])
16     rows_out.append((sum(v["worse"] for v in c.values()), c["mean_queue"]["rel"], c["energy_kwh"]["rel"], over, c))
17 rows_out.sort(key=lambda r: (r[0], r[1]))
18 for w, q, e, over, c in rows_out[:6]:
19     print(f"worse {w} queue {100*q:+.0f}% energy {100*e:+.1f}% node_hours {100*c['node_hours']['rel']:+.1f}% wear {
100*c['node_start_stop']['rel']:+.0f}% healthy {100*c['time_healthy']['rel']:+.1f}% avail {100*c['availability']['rel']:+.2f}% {over}")
20 best = min(rows_out, key=lambda r: r[1]); print("lowest queue:", f"{100*best[1]:+.0f}%", "energy", f"{100*best[2]:+.1f}%", best[3])

```

```

==== tuning/test_v2.py (29 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  import sys, json
5  from pathlib import Path
6  ROOT = Path(__file__).resolve().parents[1]; sys.path.insert(0, str(ROOT))
7  import benchmarks.stack_benchmark as sb
8  from tuning.tune_stack import scenarios, run_arm, compare, DEV_SEEDS, AllocationLaw
9  from tuning import law_v2
10 n = int(sys.argv[1]) if len(sys.argv) > 1 else 40
11 sets = [scenarios(n, s) for s in DEV_SEEDS]
12 k8s = [sb.simulate(s, "k8s_ref_70", cfg, AllocationLaw()) for cfg, scns in sets for s in scns]
13 cand = {"B_frozen": ("omni_k8s_throughput", {}), "B_tuned": ("omni_k8s_throughput", {'down_dwell': 20, 'down_after_add': 3}),
14         "C_thr_tuned": ("omni_direct_throughput", {'down_band': 8, 'down_dwell': 10, 'down_after_add': 3, 'U_gate': 0.
15         9, 'cap_min': 0.65, 'rho0': 0.85})}
16 res = {}
17 for v2 in (False, True):
18     un = law_v2.install(sb) if v2 else (lambda: None)
19     try:
20         for name, (arm, over) in cand.items():
21             rows = [r for cfg, scns in sets for r in run_arm(arm, over, cfg, scns)]
22             res[name + ("_v2" if v2 else "")] = compare(k8s, rows)
23     finally:
24         un()
25 keys = list(next(iter(res.values())).keys())
26 print(f"{'gauge':22}" + "".join(f"{'k':>16}" for k in res))
27 for m in keys:
28     print(f"{'m':22}" + "".join(f"{'100*res[k][m]'}{'rel':+14.1f}%{'!' if res[k][m]['worse'] else ' '}" for k in res))
29 print("worse count:", {k: sum(v['worse'] for v in r.values()) for k, r in res.items()})
30 json.dump(res, open(ROOT / "tuning/V2_DEV.json", "w"), indent=1)

```

```

==== tuning/tune_stack.py (122 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Tune allocation-law variants that remove the negatives of the pre-registered stack benchmark, on DEVELOPMENT seeds o
   nly.
5
6  The frozen engine files are not edited (verify.py fingerprints them). Variants are law overrides applied on top of the
7  frozen modes:
8      B (omni_k8s_throughput, Omni on top of Kubernetes): overrides applied after the throughput mode
9      C (omni_direct, Omni direct): overrides applied to the base law
10 Objective, against k8s_ref_70 on the same scenarios: energy at least 10% lower, time healthy not lower, and as few
11 gauges worse than Kubernetes as possible (wear, queue, node-hours, reversals first). Development seeds 1000 and 2000 ar
   e
12 the seeds the original constants were selected on; held-out seeds are never touched here.
13
14 python tuning/tune_stack.py --scenarios 60 --out tuning/SEARCH.json
15 """
16 from __future__ import annotations
17
18 import argparse, itertools, json, sys
19 from dataclasses import replace
20 from pathlib import Path
21
22 import numpy as np
23
24 ROOT = Path(__file__).resolve().parents[1]
25 sys.path.insert(0, str(ROOT))
26 import benchmarks.stack_benchmark as sb
27 from omnicompass import stack_sim as S
28 from omnicompass.adapter import AllocationLaw, mode_law as frozen_mode_law
29
30 DEV_SEEDS = (1000, 2000)
31 LOWER = ["energy_kwh", "node_start_stop", "machine_round_trips", "scale_reversals", "node_hours", "idle_node_hours",
32          "mean_queue", "p95_queue", "violation_backlog", "recovery_minutes", "power_cap_travel", "invariant_violations"
33          ,
34          "sla_violation_total", "pages"]
35 HIGHER = ["time_healthy", "availability", "recovered"]
36 PRIORITY = {"node_start_stop", "machine_round_trips", "scale_reversals", "mean_queue", "p95_queue", "node_hours",
37             "idle_node_hours", "violation_backlog", "availability"}
38
39 def scenarios(n, seed):
40     cfg = S.ManagerBenchmarkConfig(profile="full", scenarios=n, steps=72)
41     return cfg, S._mom_generate_scenarios(cfg, seed)[:n]
42
43
44 def run_arm(arm, over, cfg, scns):
45     base = AllocationLaw()
46     if arm == "omni_direct_throughput":
47         sb.mode_law = frozen_mode_law
48         law = replace(frozen_mode_law("throughput", base), **over)
49         try:
50             return [sb.simulate(s, "omni_direct", cfg, law) for s in scns]
51         finally:
52             sb.mode_law = frozen_mode_law
53     if arm == "omni_k8s_throughput":
54         sb.mode_law = lambda m, law: replace(frozen_mode_law(m, law), **over)
55         law = base
56     else:
57         sb.mode_law = frozen_mode_law
58         law = replace(base, **over)
59     try:
60         return [sb.simulate(s, arm, cfg, law) for s in scns]
61     finally:
62         sb.mode_law = frozen_mode_law
63
64
65 def compare(k8s, cand):
66     out = {}
67     for m in LOWER + HIGHER:
68         a = np.mean([r[m] for r in k8s]); c = np.mean([r[m] for r in cand])
69         rel = (c - a) / abs(a) if a else (0.0 if c == 0 else float("inf"))
70         worse = (rel > 0.02) if m in LOWER else (rel < -0.002)
71         out[m] = {"k8s": float(a), "cand": float(c), "rel": float(rel), "worse": bool(worse)}
72     return out
73
74
75 def score(cmp):
76     energy = cmp["energy_kwh"]["rel"]
77     ok = energy <= -0.10 and not cmp["time_healthy"]["worse"]

```

```

78     n_worse = sum(1 for v in cmp.values() if v["worse"]); n_pri = sum(1 for k, v in cmp.items() if v["worse"] and k in
PRIORITY)
79     return (0 if ok else 1, n_pri, n_worse, energy)
80
81
82 GRIDS = {
83     # round 2: the cap moves only when the engine has converged (U_gate) and with a small margin; node release held lon
ger
84     "omni_k8s_throughput": dict(down_dwell=[20, 30], down_after_add=[3, 8], U_gate=[0.45, 0.7, 0.9], margin=[0.006, 0.0
5],
85                                cap_min=[0.65, 0.8, 0.9], size_at_full_cap=[True, False]),
86     # C in throughput mode (the site power envelope is not forced), node release and target utilisation tuned
87     "omni_direct_throughput": dict(down_band=[1, 4, 8], down_dwell=[10, 20], down_after_add=[3, 8], U_gate=[0.45, 0.9],
88                                cap_min=[0.65, 0.8, 0.9], rho0=[0.914, 0.85]),
89 }
90
91
92 def main(argv=None):
93     ap = argparse.ArgumentParser(); ap.add_argument("--scenarios", type=int, default=60); ap.add_argument("--out", defa
ult="tuning/SEARCH.json")
94     a = ap.parse_args(argv)
95     sets = [scenarios(a.scenarios, s) for s in DEV_SEEDS]
96     k8s = [sb.simulate(s, "k8s_ref_70", cfg, AllocationLaw()) for cfg, scns in sets for s in scns]
97     result = {"dev_seeds": DEV_SEEDS, "scenarios_per_seed": a.scenarios, "arms": {}}
98     for arm, grid in GRIDS.items():
99         keys = list(grid); trials = []
100         frozen = [r for cfg, scns in sets for r in run_arm(arm, {}, cfg, scns)]
101         fc = compare(k8s, frozen)
102         for vals in itertools.product(*grid.values()):
103             over = dict(zip(keys, vals))
104             rows = [r for cfg, scns in sets for r in run_arm(arm, over, cfg, scns)]
105             c = compare(k8s, rows)
106             trials.append({"overrides": over, "score": list(score(c)), "compare": c})
107             trials.sort(key=lambda t: t["score"])
108         result["arms"][arm] = {"frozen": {"score": list(score(fc)), "compare": fc}, "best": trials[0], "top5": trials[:
5], "trials": len(trials)}
109         b = trials[0]
110         print(f"== {arm}: {len(trials)} variants; frozen worse gauges {sum(v['worse'] for v in fc.values())}, "
111               f"best worse gauges {sum(v['worse'] for v in b['compare'].values())}")
112         print("  best overrides:", b["overrides"])
113         for m, v in b["compare"].items():
114             flag = "WORSE" if v["worse"] else ""
115             print(f"    {m:22} k8s {v['k8s']:10.3f} frozen {fc[m]['cand']:10.3f} ({100 * fc[m]['rel']:6.1f}%) "
116                   f"tuned {v['cand']:10.3f} ({100 * v['rel']:6.1f}%) {flag}")
117         Path(a.out).parent.mkdir(parents=True, exist_ok=True)
118         Path(a.out).write_text(json.dumps(result, indent=1))
119
120
121 if __name__ == "__main__":
122     main()

```

```

==== tuning/vendor_compare.py (31 lines) ====
1  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
2  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
3  # Omni-Compass Enterprise License. See LICENSE.
4  """Named products vs Omni-Compass on the fleet plant with the response-time gauge (fleet/sim_slo.py), development seeds
   .
5  Vendor arms emulate documented behaviour (see fleet/sim_slo.py VENDOR notes); they are not the vendors' binaries."""
6  import sys, json; sys.path.insert(0, '.')
7  import numpy as np
8  from multiprocessing import Pool
9  from fleet import sim_slo; from fleet.harness import make_scenario
10 from tuning.speed_search import gauges, LOWER, HIGHER
11 from omnicompass.speed import SpeedLaw
12 R=json.load(open('tuning/SPEED_SEARCH_21.json'))
13 best={r['i']:r['cfg'] for r in R}
14 ARMS=['k8s_hpa70_ca', 'openshift', 'gke_optimize', 'aks_nap', 'turbonomic', 'omni_fleet', 'speed176', 'speed159']
15 def one(k):
16     v,s=k; sc=make_scenario(v,s); out={}
17     for a in ARMS:
18         if a.startswith('speed'):
19             c=best[int(a[5:])]; r=sim_slo.run(sc, 'omni_speed', speed_law=SpeedLaw(**c['law']), omni_every=c['every'], lat_
gain=c['lat_gain'], slo_mult=c['slo_mult'])
20             else: r=sim_slo.run(sc,a)
21             out[a]=gauges(r)
22     return k,out
23 if __name__=='__main__':
24     keys=[(v,s) for v in ('web', 'multi', 'batch', 'gpu') for s in (101,102,103,104)]
25     with Pool(4) as p: res=dict(p.map(one,keys))
26     out={v:{a:{m:float(np.mean([res[(v,s)][a][m] for s in (101,102,103,104))]) for m in LOWER+HIGHER} for a in ARMS} fo
r v in ('web', 'multi', 'batch', 'gpu')}
27     open('tuning/VENDOR_COMPARE_DEV.json', 'w').write(json.dumps(out, indent=1))
28     for v in ('web', 'multi', 'batch', 'gpu'):
29         print(v, ' '.join(f'{a[:11]:>12}' for a in ARMS))
30         for m in ['energy_kwh', 'node_hours', 'p95_ms', 'p99_ms', 'mean_ms', 'start_stop', 'node_reversals', 'work_completed',
'time_healthy']:
31             print(f' {m:15}'+'.join(f'{np.mean([res[(v,s)][a][m] for s in (101,102,103,104))}:12.4g}' for a in ARMS))

```

```

==== verify.py (340 lines) ====
1  #!/usr/bin/env python3
2  # SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
3  # Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
4  # Omni-Compass Enterprise License. See LICENSE.
5  """One-command verification of the Omni-Compass package.
6
7  python verify.py          full verification (about 15-30 minutes, single core)
8  python verify.py --full-replay  additionally re-runs all held-out scenarios for every arm and requires the
9                                  regenerated RUNS.csv and SUMMARY.json to equal the shipped files
10 python verify.py --quick    skips the 20,000-case engine comparison, runs a 1,000,000-decision soak
11                                instead of 100,000,000, and replays fewer scenarios
12 """
13 import argparse, csv, hashlib, json, shutil, subprocess, sys, tempfile
14 from pathlib import Path
15 ROOT = Path(__file__).resolve().parent
16 sys.path.insert(0, str(ROOT))
17 ok = True
18
19 def check(name, cond, detail=""):
20     global ok
21     print(("PASS " if cond else "FAIL ") + name + {" ({"detail"})" if detail else ""})
22     ok = ok and bool(cond)
23
24 def sha(p): return hashlib.sha256(Path(p).read_bytes()).hexdigest()
25
26 def build_cpp(tmp):
27     src = ROOT / "cpp"; b = tmp / "build"
28     if shutil.which("cmake"):
29         subprocess.run(["cmake", "-S", str(src), "-B", str(b), "-DCMAKE_BUILD_TYPE=Release"], check=True, capture_output=True)
30     else:
31         subprocess.run(["cmake", "--build", str(b), "-j2"], check=True, capture_output=True)
32     b.mkdir(parents=True)
33     cxx = shutil.which("g++") or shutil.which("clang++")
34     srcs = [str(p) for p in (src / "src").glob("*.cpp")]
35     for tool, name in (("run_fixture.cpp", "oc_run_fixture"), ("run_governor.cpp", "oc_governor"), ("soak.cpp", "oc_soak"),
36 ("smoke.cpp", "oc_smoke"), ("run_shield.cpp", "oc_shield"), ("run_hpa.cpp", "oc_hpa"), ("savings.cpp", "oc_savings"),
37 ("run_closure.cpp", "oc_closure"), ("run_conveyance.cpp", "oc_conveyance"), ("run_twins.cpp", "oc_twins")):
38         subprocess.run([cxx, "-std=c++20", "-O2", "-I", str(src / "include"), *srcs, str(src / "tools" / tool), "-o", str(b / name)], check=True)
39     return b
40
41 def main():
42     ap = argparse.ArgumentParser(); ap.add_argument("--quick", action="store_true"); ap.add_argument("--full-replay", action="store_true")
43     a = ap.parse_args()
44     sys.path.insert(0, str(ROOT / "tools"))
45     from code_fingerprint import fingerprint
46     prov = json.loads((ROOT / "reference" / "PROVENANCE.json").read_text())
47     ref = ROOT / prov["reference_engine"]
48     import platform
49     pyver = ".".join(platform.python_version_tuple()[:2])
50     check("reference engine SHA-256", sha(ref) == prov["reference_engine_sha256"])
51     reg = json.loads((ROOT / "results" / "PREREGISTRATION.json").read_text())
52     amend = {a["file"]: a for a in json.loads((ROOT / "results" / "LOCK_AMENDMENTS.json").read_text())["amendments"]}
53
54     def locked(f, h): # unchanged, or changed exactly as a recorded amendment (bug fix) says
55         cur = sha(ROOT / f)
56         if cur == h:
57             return True, ""
58         a = amend.get(f)
59         ok = a is not None and h in a["from_sha256"] and cur == a["to_sha256"]
60         return ok, (f"amended in {a['commit']}: {a['reason'][:60]}..." if ok else "")
61     for f, h in reg["sha256"].items():
62         ok, note = locked(f, h)
63         check(f"pre-registered file unchanged or amended on record (SHA-256): {f}", ok, note)
64     if pyver == reg.get("fingerprint_python", ""):
65         check("reference engine program fingerprint", fingerprint(ref) == prov["program_fingerprint"])
66         for f, h in reg["code_fingerprint"].items():
67             check(f"program fingerprint: {f}", fingerprint(ROOT / f) == h)
68     else:
69         print(f"SKIP program fingerprints (recorded with Python {reg.get('fingerprint_python')}, running {pyver}); "
70               "ast.dump output is interpreter-version dependent; SHA-256 checks above are authoritative")
71     from tests import test_stack_sim_provenance, test_native_equivalence, test_cpp_governor_parity
72     test_stack_sim_provenance.main(); check("stack simulator provenance", True)
73     if not a.quick:
74         from tests import test_core_parity

```

```

75     test_core_parity.main(); check("core parity vs reference engine and 500 fixtures", True)
76     test_native_equivalence.main(20 if a.quick else 60); check("native arm equals reference simulator", True)
77     tmp = Path(tempfile.mkdtemp()); b = build_cpp(tmp)
78     subprocess.run([str(b / "oc_run_fixture"), str(ROOT / "fixtures" / "current_500_inputs.csv"), str(tmp / "cpp.csv")]
, check=True)
79     r = subprocess.run([sys.executable, str(ROOT / "cpp" / "scripts" / "compare_results.py"), str(ROOT / "fixtures" / "
current_500_expected.csv"),
80                       str(tmp / "cpp.csv"), str(tmp / "cpp.json")], capture_output=True, text=True)
81     rep = json.loads((tmp / "cpp.json").read_text())
82     check("C++ core vs 500 fixtures", rep["pass"], f"max abs {rep['max_abs_error']:.2e}")
83     test_cpp_governor_parity.main(str(b / "oc_governor"), 20 if a.quick else 60); check("C++ governor vs Python governo
r (power_protect)", True)
84     test_cpp_governor_parity.main(str(b / "oc_governor"), 20 if a.quick else 60, "throughput"); check("C++ governor vs
Python governor (throughput)", True)
85     for m_ in ("fleet", "fleet_balanced", "fleet_wear"):
86         test_cpp_governor_parity.main(str(b / "oc_governor"), 20 if a.quick else 60, m_); check(f"C++ governor vs Pytho
n governor ({m_})", True)
87     from tests import test_cpp_shield_parity
88     test_cpp_shield_parity.main(str(b / "oc_shield"), 8 if a.quick else 20); check("C++ shield vs Python shield (enforc
e and violations)", True)
89     from tests import test_cpp_shield_adversarial, test_shield_properties
90     test_shield_properties.main(100_000 if a.quick else 1_000_000); check("shield properties: no invariant violated, id
empotent, no invented action, minimal intervention (random + adversarial)", True)
91     test_cpp_shield_adversarial.main(str(b / "oc_shield"), 50_000 if a.quick else 200_000); check("C++ shield vs Python
shield on the adversarial generator", True)
92     from tests import test_cpp_closure_parity
93     test_cpp_closure_parity.main(str(b / "oc_closure"), 1 if a.quick else 3); check("C++ closure law vs Python closure
law, every decision and reserve", True)
94     from tests import test_cpp_conveyance_parity
95     w = test_cpp_conveyance_parity.main(str(b / "oc_conveyance"), 50 if a.quick else 200)
96     check("C++ conveyance law vs Python conveyance law, every allocation (random systems and the CPU+GPU layout)", w <
1e-9)
97     from tests import test_cpp_twins_parity
98     test_cpp_twins_parity.main(str(b / "oc_twins"), 500 if a.quick else 2000)
99     check("C++ nervous system, compass and ledger, GPU rules vs Python, value for value", True)
100    from tools import seal
101    broken = seal.check()
102    check(f"seal: all {len(seal.TWINS)} Python/C++ twins unchanged since proven equal (results/SEAL.json)", not broken,
"; ".join(broken))
103    from tools import mechanism_identity, tracking_bounds
104    bad = mechanism_identity.check()
105    check("mechanism identity: F, Theta, C, h, G, M_act, dt, A of both configurations match results/MECHANISM_IDENTITY.
json", not bad, "; ".join(bad))
106    tb = tracking_bounds.main() # recomputed; a changed result also shows as a manifest mismatch below
107    nt = tb["nearest_target"]
108    check("tracking theorem checks (V): drift under F_bar < U_AUTHORITY, discrete ultimate bound inside the basin, no i
nvariance failure",
109          nt["drift_within_bound_all"] and nt["F_bar_below_U_AUTHORITY_all"] and nt["ultimate_bound_inside_basin"]
110          and nt["invariance_failures"] == 0 and tb["wrong_target"]["invariance_failures"] == 0)
111    from tools import release_manifest
112    bad = release_manifest.check()
113    check("release manifest: engine, C++ seal, GPU protocol, live evidence and license match RELEASE_MANIFEST.json", no
t bad, "; ".join(bad))
114    src = (ROOT / "cpp" / "src" / "shield.cpp").read_text()
115    mut_src = tmp / "shield_mut.cpp"
116    mut_src.write_text(src.replace('return k == "nodes" || k == "terraform_plan" || k == "rollout"; }', 'return k == "n
odes" || k == "terraform_plan"; }'))
117    others = [str(p) for p in (ROOT / "cpp" / "src").glob("*.cpp") if p.name != "shield.cpp"]
118    subprocess.run([shutil.which("g++") or shutil.which("clang++"), "-std=c++20", "-O2", "-I", str(ROOT / "cpp" / "incl
ude"), *others, str(mut_src),
119                  str(ROOT / "cpp" / "tools" / "run_shield.cpp"), "-o", str(tmp / "oc_shield_mut")], check=True)
120    try:
121        test_cpp_shield_parity.main(str(tmp / "oc_shield_mut"), 8)
122        det3 = False
123    except AssertionError:
124        det3 = True
125    check("negative control: parity test detects a C++ shield that no longer blocks rollouts during a security block",
det3)
126    from tests import test_cpp_hpa_parity
127    test_cpp_hpa_parity.main(str(b / "oc_hpa"), (901,) if a.quick else (901, 902)); check("independent C++ HPA replica
law vs fleet harness HPA", True)
128    hsrc = (ROOT / "cpp" / "include" / "omnicompass" / "hpa.hpp").read_text()
129    hm = tmp / "hmut" / "omnicompass"; hm.mkdir(parents=True)
130    for p_ in (ROOT / "cpp" / "include" / "omnicompass").glob("*.hpp"):
131        (hm / p_.name).write_text(p_.read_text())
132        (hm / "hpa.hpp").write_text(hsrc.replace("tolerance{0.1}", "tolerance{0.05}"))
133    subprocess.run([shutil.which("g++") or shutil.which("clang++"), "-std=c++20", "-O2", "-I", str(tmp / "hmut"), str(R
OOT / "cpp" / "src" / "hpa.cpp"),
134                  str(ROOT / "cpp" / "tools" / "run_hpa.cpp"), "-o", str(tmp / "oc_hpa_mut")], check=True)
135    try:
136        test_cpp_hpa_parity.main(str(tmp / "oc_hpa_mut"), (901,))

```

```

137     det4 = False
138     except AssertionError:
139         det4 = True
140     check("negative control: parity test detects a C++ HPA with 5% instead of 10% tolerance", det4)
141     r = subprocess.run([str(b / "oc_smoke"), capture_output=True, text=True)
142     check("C++ governor contract smoke test (bounds, security, determinism, both modes)", r.returncode == 0 and "SMOKE
PASS" in r.stdout)
143     n_soak = 1000000 if a.quick else 100000000
144     rec_all = json.loads((ROOT / "results" / "SOAK.json").read_text())
145     for mode in ("power_protect", "throughput"):
146         soak = json.loads(subprocess.run([str(b / "oc_soak"), str(n_soak), mode], capture_output=True, text=True).stdou
t)
147         cps = soak["rss_kb_at_25_50_75_100pct"]
148         check(f"soak test ({mode}), {n_soak:,} decisions, no failures, memory flat after warm-up", soak["nonfinite_or_o
ut_of_range"] == 0 and max(cps) == min(cps),
149             f"{soak['ns_per_decision']:.0f} ns per decision, RSS checkpoints {cps} kB")
150         if not a.quick:
151             rec = rec_all[mode]
152             same = all(soak[k] == rec[k] for k in ("decisions", "nonfinite_or_out_of_range", "E", "U", "I_U", "S", "B"))
153         check(f"soak test ({mode}) reproduces results/SOAK.json (decisions, failures, state ranges)", same)
154         hdr = (ROOT / "cpp" / "include" / "omnicompass" / "governor.hpp").read_text()
155         mut = tmp / "mut" / "omnicompass"; mut.mkdir(parents=True)
156         for src_h in (ROOT / "cpp" / "include" / "omnicompass").glob("*.hpp"): # every header, so new modules compile too
157             (mut / src_h.name).write_text(src_h.read_text())
158             (mut / "governor.hpp").write_text(hdr.replace("down_dwell{4}", "down_dwell{3}"))
159             cxx = shutil.which("g++") or shutil.which("clang++")
160             subprocess.run([cxx, "-std=c++20", "-O2", "-I", str(tmp / "mut"), *[str(p) for p in (ROOT / "cpp" / "src").glob("*.
cpp")],
161                             str(ROOT / "cpp" / "tools" / "run_governor.cpp"), "-o", str(tmp / "oc_mut")], check=True)
162             try:
163                 test_cpp_governor_parity.main(str(tmp / "oc_mut"), 20)
164                 detected = False
165             except AssertionError:
166                 detected = True
167             check("negative control: parity test detects a mutated C++ governor (dwell 3 instead of 4)", detected)
168             (mut / "governor.hpp").write_text(hdr.replace("push_release{0.005}", "push_release{9.0}"))
169             subprocess.run([cxx, "-std=c++20", "-O2", "-I", str(tmp / "mut"), *[str(p) for p in (ROOT / "cpp" / "src").glob("*.
cpp")],
170                             str(ROOT / "cpp" / "tools" / "run_governor.cpp"), "-o", str(tmp / "oc_mut2")], check=True)
171             try:
172                 test_cpp_governor_parity.main(str(tmp / "oc_mut2"), 20)
173                 detected2 = False
174             except AssertionError:
175                 detected2 = True
176             check("negative control: parity test detects a C++ governor without the equation (2) release gate", detected2)
177             (mut / "governor.hpp").write_text(hdr.replace("push_add{-9.0}", "push_add{9.0}"))
178             subprocess.run([cxx, "-std=c++20", "-O2", "-I", str(tmp / "mut"), *[str(p) for p in (ROOT / "cpp" / "src").glob("*.
cpp")],
179                             str(ROOT / "cpp" / "tools" / "run_governor.cpp"), "-o", str(tmp / "oc_mut3")], check=True)
180             try:
181                 test_cpp_governor_parity.main(str(tmp / "oc_mut3"), 20)
182                 detected3 = False
183             except AssertionError:
184                 detected3 = True
185             check("negative control: parity test detects a C++ governor with a wrong equation (2) add gate", detected3)
186             from benchmarks.stack_benchmark import simulate, ARMS
187             from omnicompass import stack_sim as S
188             from omnicompass.adapter import AllocationLaw
189             n = 5 if a.quick else 20
190             for sd in reg["held_out_seeds"]:
191                 rows = list(csv.DictReader(open(ROOT / "results" / f"heldout_seed_{sd}" / "RUNS.csv")))
192                 cfg = S.ManagerBenchmarkConfig(profile="full", scenarios=500, steps=72)
193                 scns = S._mom_generate_scenarios(cfg, sd)[:n]; bad = 0
194                 for s in scns:
195                     for arm in ARMS:
196                         new = simulate(s, arm, cfg, AllocationLaw())
197                         old = next(r for r in rows if int(r["scenario_id"]) == s.scenario_id and r["arm"] == arm)
198                         bad += int(new["trace_hash"] != old["trace_hash"] or abs(new["time_healthy"] - float(old["time_healthy"]
])) > 1e-12)
199             check(f"held-out seed {sd}: {n} scenarios x {len(ARMS)} arms replay identically", bad == 0)
200             summ = json.loads((ROOT / "results" / f"heldout_seed_{sd}" / "SUMMARY.json").read_text())
201             check(f"held-out seed {sd}: observe mode identical to native", summ["observe_mode_identical_to_native"] == summ
["scenarios"])
202             for arm in ("omni_k8s_protect", "omni_direct", "omni_direct_no_dynamics"):
203                 check(f"held-out seed {sd}: {arm} invariant violations I1-I5 = 0", summ["means"][arm]["invariant_violations
"] == 0.0)
204             check(f"held-out seed {sd}: omni_k8s_throughput invariant violations I1-I3, I5 = 0 (I4 not enforced in throughp
ut mode)",
205                 summ["means"]["omni_k8s_throughput"]["invariant_violations_ex_power"] == 0.0)
206             check(f"held-out seed {sd}: layered observe mode identical to Kubernetes alone",

```

```

207         summ["layered_observe_identical_to_kubernetes"] == summ["scenarios"])
208     freg = json.loads((ROOT / "results" / "fleet" / "PREREGISTRATION.json").read_text())
209     for f, h in freg["sha256"].items():
210         if f != "omnicompass/adapters.py":
211             ok, note = locked(f, h)
212             check(f"fleet pre-registered file unchanged or amended on record (SHA-256): {f}", ok, note)
213     from fleet.sim import run as frun, arms_for
214     from fleet.harness import make_scenario
215     frows = list(csv.DictReader(open(ROOT / "results" / "fleet" / "heldout" / "RUNS.csv")))
216     fsumm = json.loads((ROOT / "results" / "fleet" / "heldout" / "SUMMARY.json").read_text())
217     for v in ("web", "multi", "batch", "gpu", "gpu_always_on"):
218         seeds = range(freg["held_out_seed_base"], freg["held_out_seed_base"] + (2 if a.quick else 5))
219         bad = 0
220         for s_ in seeds:
221             scn = make_scenario(v, s_)
222             for arm in arms_for(v):
223                 new = frun(scn, arm)
224                 old = next(r for r in frows if r["vessel"] == v and int(r["seed"]) == s_ and r["arm"] == arm)
225                 bad += int(new["trace_hash"] != old["trace_hash"] or abs(float(new["energy_kwh"]) - float(old["energy_kwh"]))) > 1e-9)
226             check(f"fleet held-out ({v}): {len(seeds)} scenarios x {len(arms_for(v))} arms replay identically", bad == 0)
227             check(f"fleet ({v}): observe mode identical to HPA 0.7 + Cluster Autoscaler",
228                   fsumm["vessels"][v]["observe_identical_to_hpa70_ca"] == fsumm["vessels"][v]["scenarios"])
229     from fleet import planetlab as PLm
230     pl_rows = list(csv.DictReader(open(ROOT / "results" / "fleet" / "planetlab" / "RUNS.csv")))
231     tr = PLm.load_dir(ROOT / "fleet" / "traces" / "planetlab"); badp = 0
232     for s_ in range(800000, 800000 + (1 if a.quick else 3)):
233         scn = PLm.make_scenario(tr, s_)
234         for arm in arms_for("web"):
235             new = frun(scn, arm); old = next(r for r in pl_rows if int(r["seed"]) == s_ and r["arm"] == arm)
236             badp += int(new["trace_hash"] != old["trace_hash"])
237             check("recorded-trace (PlanetLab) runs replay identically", badp == 0)
238     tp = json.loads((ROOT / "fleet" / "traces" / "PROVENANCE.json").read_text())
239     check("recorded traces match their registered SHA-256", all(hashlib.sha256((ROOT / "fleet" / "traces" / "planetlab" / n).read_bytes()).hexdigest() == v["sha256"] for n, v in tp["files"].items()))
240     from tests import test_hpa_three_way
241     test_hpa_three_way.main((901,) if a.quick else (901, 902)); check("three-way HPA agreement (harness, C++, external implementation with upstream window)", True)
242     import benchmarks.savings as SV
243     sv_tmp = tmp / "savings"; sv_tmp.mkdir()
244     SV.main(str(b / "oc_savings"), sv_tmp)
245     check("C++ savings projector matches Python reference and reproduces results/SAVINGS.csv",
246           (sv_tmp / "SAVINGS.csv").read_text() == (ROOT / "results" / "SAVINGS.csv").read_text())
247     if not a.quick:
248         from tests import test_selfpilot
249         test_selfpilot.main(); check("end-to-end self-pilot: shipped controller, simulated cluster, capture and scoring (energy and machines no worse, no significant pending-pod increase)", True)
250         from tests import test_pilot_score
251         test_pilot_score.main(); check("pilot scoring: detects a real gain, no false gain on identical clusters, detects a service regression", True)
252         from tests import test_omni_controller
253         test_omni_controller.main(); check("live controller against a fake cluster: observe writes nothing, target bounded, kill restores, node pool bounded and dry-run safe", True)
254         from tests import test_bowl_controller
255         test_bowl_controller.main(); check("live controller, bowl law: never tighter than native, one machine back per decision through the gate, fail up past the wall and when blind, kill restores, the engine runs every decision", True)
256         from tests import test_muscles
257         test_muscles.main(); check("live muscles: power cap, heat, security, rollout, batch, CPU frequency and GPU connectors; kill restores; observe writes nothing", True)
258         from tests import test_nervous_system
259         test_nervous_system.main(50_000 if a.quick else 300_000); check("nervous system: observe executes nothing, kill removes authority, security blocks capacity expansion, stress never grants contraction, envelopes in hardware range, deterministic", True)
260         from tests import test_conveyance
261         test_conveyance.main(); check("conveyance law: budget conserved, Lyapunov ascent, exact exponential convergence, bounds, budget never exceeded", True)
262         from tests import test_two_way
263         test_two_way.main(); check("two-way nervous system: blind senses never grant contraction, frozen probe detected, failures read as pressure, no new order before the last one landed", True)
264         from tests import test_node_release_gate
265         test_node_release_gate.main(); check("machine-organ release gate: own engine view, pods-first coordination, headroom at the engine's rho", True)
266         from tests import test_hardware_plant
267         test_hardware_plant.main(); check("hardware plant: GPU arms B and C distinct; C = min(engine cap, (want/rho)^(1/gamma)); no lost work, no extra heat", True)
268         from tests import test_node_exchange
269         test_node_exchange.main(); check("CPU+GPU on one budget: never over the site budget; CPU-measured-only goes over; more work than today's fixed cap", True)
270         from tests import test_cpufreq_contract
271         test_cpufreq_contract.main(); check("CPU-frequency lever via kernel policy files: schedutil floor, nervous envelope, never above the operator ceiling, exact restore on kill", True)

```

```

272     from tests import test_strict_replicas
273     test_strict_replicas.main(); check("strict C live mode: Omni-Compass sets the replica floor, growth free, kill rest
ores the range", True)
274     from tests import test_compass
275     test_compass.main(); check("the compass: face, axle, closed circles, inwardness on the boundary, ledger descent", T
rue)
276     from tests import test_living_band
277     test_living_band.main(); check("living band: every level the nervous system hands out stays inside 5%..95%; budget
kept", True)
278     from tests import test_pod_reflex
279     test_pod_reflex.main(); check("fast pod reflex: the HPA's own rule read from the live queue; calm writes nothing; h
ands back; kill restores", True)
280     from tests import test_muscles_levers
281     test_muscles_levers.main(); check("live levers: rightsize, coldstart, batch pace, containment, cooling; kill restor
es every one, also from a fresh process", True)
282     from tests import test_pod_starts
283     test_pod_starts.main(); check("exact pod-start timing from the API server's watch stream (creation to Ready, window
-bounded)", True)
284     from tests import test_gpu_bench
285     test_gpu_bench.main(); check("GPU bench: watch writes nothing, limit never below draw x 1.3, read-back, blind and S
LO reflexes, kill restores; paired run validity", True)
286     from tests import test_gpu_bowl
287     test_gpu_bowl.main(); check("two-wire GPU governor: races while work waits, never slower than the card on its own w
hile busy, lid never under its own busy draw, fail up past the line, restores", True)
288     from tests import test_cost_to_match
289     test_cost_to_match.main(); check("cost to match: the cheapest native setting that reaches Omni-Compass's p95, its e
xtra pods, CPU and machines", True)
290     from tests import test_server_power
291     test_server_power.main(); check("server power: Redfish and IPMI give the whole-server watts, read only", True)
292     from tests import test_disruption_budget, test_muscle_sdk, test_causal_evidence_gate, test_gpu_watchdog
293     test_disruption_budget.main(); check("disruption budget: node moves capped per period, the score charges every disr
uption", True)
294     test_muscle_sdk.main(); check("universal muscle SDK: a customer's knob plugs in with bounds, authority and a restor
e point", True)
295     test_causal_evidence_gate.main(); check("causal evidence gate: a claim is held to the evidence level its authority
supports", True)
296     test_gpu_watchdog.main(); check("independent GPU watchdog: a governor killed outright, the card's start limit resto
red", True)
297     from tests import test_api_proxy
298     test_api_proxy.main(); check("controller reads through one kubectrl proxy: kubectrl's own output shape, missing objec
ts are errors, anything else goes to kubectrl", True)
299     from tests import test_master_switch
300     test_master_switch.main(); check("master switch: one OFF stops every governor at once and hands every setting back;
nothing starts while OFF; ON allows a start", True)
301     from tests import test_verdict
302     test_verdict.main(); check("verdict: a knob moves only where a paired trial shows the muscle no worse than the allo
wance; left native where every step costs", True)
303     from tests import test_convey
304     test_convey.main(); check("energy to where the work is: idle machine CPU conveyed to serving pods in place, band-bo
unded, kill restores", True)
305     from tests import test_active_nodes
306     test_active_nodes.main(); check("full-engine controller options: parked and control-plane nodes excluded, kill swit
ch restores the node pool once", True)
307     from tests import test_schedutil, test_cpufreq_ceiling
308     test_schedutil.main(); check("schedutil model: 1.25 map tips at 80%, OPP snap, uclamp, RT to policy max, rate limit
, iowait boost, Omni ceiling and kill", True)
309     test_cpufreq_ceiling.main(); check("cpufreq ceiling writer: scaling_max_freq on every policy, clamped; restore puts
cpuinfo_max_freq back", True)
310     from tests import test_realms
311     test_realms.main(); check("realm harness: catalog of 656, watch equals native, kill hands back every knob, determin
istic, capacity law, labels", True)
312     from tests import test_bowl
313     test_bowl.main(); check("bowl law and plug: smooth bounded push and pull to the center, fail up, cover, one restore
point, foreign writer, two-wire card", True)
314     from tests import test_failsafe
315     test_failsafe.main(); check("no automated fallback: failed decisions are skipped; only the human switch turns the w
hole harness off and on", True)
316     from tests import test_fleet_realdatal_paths
317     test_fleet_realdatal_paths.main(); check("capture replay and PlanetLab vessel on inputs in the real formats", True)
318     r = subprocess.run(["bash", "-n", str(ROOT / "fleet" / "capture" / "kube_capture.sh")], capture_output=True)
319     check("cluster capture script parses", r.returncode == 0)
320     import benchmarks.multiplicity as MP
321     mp_tmp = tmp / "mult.json"; MP.main(mp_tmp)
322     check("multiplicity analysis reproduces results/MULTIPLICITY.json",
323           json.loads(mp_tmp.read_text()) == json.loads((ROOT / "results" / "MULTIPLICITY.json").read_text()))
324     if a.full_replay:
325         from benchmarks.stack_benchmark import run as run_bench
326         for sd in reg["held_out_seeds"]:
327             out = tmp / f"replay_{sd}"
328             run_bench(reg["held_out_scenarios_per_seed"], sd, out, AllocationLaw())
329             same_runs = (out / "RUNS.csv").read_bytes() == (ROOT / "results" / f"heldout_seed_{sd}" / "RUNS.csv").read_

```

```
bytes()
330     new_s = json.loads((out / "SUMMARY.json").read_text()); old_s = json.loads((ROOT / "results" / f"heldout_se
ed_{sd}" / "SUMMARY.json").read_text())
331     new_law, old_law = new_s.pop("allocation_law"), old_s.pop("allocation_law")
332     inert = {"push_add": -9.0}
333     law_ok = all(new_law.get(k) == v for k, v in old_law.items()) and all(k in inert and new_law[k] == inert[k]
for k in set(new_law) - set(old_law))
334     same_summary = new_s == old_s and law_ok
335     check(f"full replay, held-out seed {sd}: all scenarios x all arms regenerate RUNS.csv exactly and SUMMARY.j
son exactly (recorded law may add only inert defaults)", same_runs and same_summary)
336     print("\nVERIFICATION:", "PASS" if ok else "FAIL")
337     sys.exit(0 if ok else 1)
338
339 if __name__ == "__main__":
340     main()
```